DDA Scattering Pipeline — Project Manual

Single scattering properties of randomly oriented hydrometeors (ADDA → ARTS SSDB)

Version: 2026-11-06 · repo state: Phase 5 complete, ARO/TRO backlog ready for production runs

---

How to use this manual

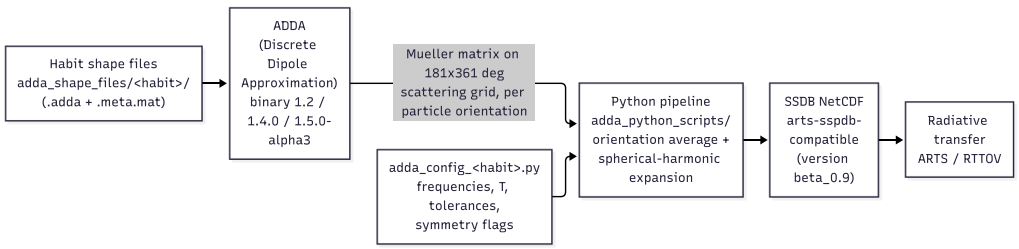
This is the single reference document for maintaining and operating the DDA automation pipeline in this repository. It is written for the **future project maintainer**: enough physics theory to understand *what* is computed and *why* each knob matters, and enough operational detail (verified against the actual source, not from memory) to run, extend and debug the pipeline safely.

- Flowcharts / call graphs are **Mermaid** diagrams — they render on GitHub, in VS Code (Markdown preview), and in the generated HTML (`docs/DDA-manual.html`).
- All quantitative plots are **real data** (measured benchmarks, real shape metadata, the actual Matzler-2006 implementation). They are regenerated by `make_figures.py`: `mamba activate DDA && python docs/make_figures.py` → `docs/figures/*.png`. If you change the backlog or run new benchmarks, re-run it and commit the figures.
- The browsable HTML version is built with `python docs/build_html.py` (add `--pdf` if a headless Chrome/Chromium is installed). Mermaid diagrams in the HTML render via mermaid.js from a CDN — same requirement as GitHub.

Table of contents

Part	Content
0	What the project computes, where things live, glossary
1	DDA, shape model & resolution scheme, ice dielectric, Mueller matrices, orientation averaging (ARO/TRO), spherical harmonics, mirror symmetry, SSDB format
2	Call graphs (ARO/TRO), artifact data flow, module status table
3	Config contract + annotated walkthrough, job flowcharts, local & HPC runbooks, idempotency, env overrides
4	Golden baselines, noise-based gates, equivalence suites, backlog status
A … C	ADDA CLI tables, input .dat files, NetCDF layout
D	Catalogue of traps, with bug-fix history

The one-minute picture



The pipeline answers one question: *for a given particle habit, size, frequency and temperature, what are its single-scattering properties averaged over all relevant particle orientations?* Two orientation strategies are produced:

Strategy	Physical situation	Orientation space covered	Output coordinates
ARO — azimuthally random oriented	cloud particle whose symmetry/maximum axis stays **horizontal** (gravity), azimuth fully random	Euler-β tilt grid × incidence-angle grid (5$\times $10solved, re-griddedto19$\times $19 in output)	one NetCDF per (β, size, f, T)

Strategy	Physical situation	Orientation space covered	Output coordinates
TRO — totally random oriented	freely tumbling particle	full Euler space (α, β, γ) — integrated away	orientation-independent cross sections + Mueller

---

Part 0 — Orientation

0.1 Repository map

```
DDA/
├── adda_python_scripts/        ← THE PIPELINE (orchestration, post-processing, export)
│   ├── adda_config_*.py        one config per habit (ALL knobs live here)
│   ├── adda_main_control.py    ARO entry point (T×f×size loop, job submission)
│   ├── adda_sub_control_spharmonics.py   one ARO job: icosphere orientation loop + ADDA
│   ├── adda_main_control_rndorient.py    TRO entry point (chunked job-table)
│   ├── adda_sub_control_randomorient.py  one TRO job: ADDA '-orient avg'
│   ├── adda_process_aro_particles*.py    post-processing (averaging, re-expansion)
│   ├── adda2SSDBnetcdf_functions.py      SSDB export writer
│   ├── definitions_for_netcdf.py         SSDB variable/dimension definitions
│   ├── adda_control_functions.py         (acf) ADDA CLI wrapper, dielectric, mirror transforms
│   ├── adda_orientational_avg_spharmonics_functions.py  (aoa) ARO orientation averaging
│   ├── spherical_harmonics_for_adda.py   (spha) shtns SH expansion machinery
│   ├── icosphere_grid.py       (ico) orientation-sampling grids
│   └── old_configs/, *_for_testing.py, adda_sub_control_gridref.py  ← ARCHIVE, do not extend
├── adda/                       ADDA 1.5.0-alpha3 source (built locally: adda/src/seq/adda)
├── adda_1.4/                   ADDA 1.4.0  ← PRODUCTION BINARY (Phase-3 decision)
├── adda_1.2/                   ADDA 1.2    (golden baselines were made with this)
├── adda_shape_files/           particle geometries: <habit>/Dmax…_Dveq…_Mass…_Freq…GHz.adda(+.meta.mat)
├── adda_aux_input/             avg_params.dat files (TRO '-orient avg' integration control)
├── arts-sspdb-interface/       upstream read-only tooling for the *published* ARTS database
├── plans/                      modernization plan + running findings report (READ before touching code)
├── baselines/                  frozen golden ADDA outputs + regression check tooling
├── analysis/                   phase analysis/gate scripts + benchmark results (JSON/TXT/LOG)
└── docs/                       this manual + figures + build scripts
```

> **Results never live in this repo.** Intermediate and final data go to `<prefix_data>/adda_results/…` (cluster) or `<repo>/adda_results/…` (local mode). The old HPC results are *not accessible* — regression validation therefore relies on `baselines/` (golden ADDA outputs) and the equivalence suites in `analysis/`.

0.2 Glossary

Term	Meaning in this project
ADDA	*Discrete Dipole Approximation* code (Yurkin & Hoekstra). Three trees in-repo: `adda_1.2` , `adda_1.4` (production), `adda/` (1.5.0-alpha3).
ARO / TRO	azimuthally random orientation (axis horizontal) / totally random orientation.
D_veq	volume-equivalent particle diameter; in result-folder names the field is spelled `deq` — and in **TRO** folders it misleadingly carries the equivalent **radius** (legacy, §D-14).
D_max	maximum particle extent (defines the size grid step naming, e.g. `Dmax00013um`).
size parameter x	$x = \pi D_{\mathrm{veq}}/\lambda = \pi D_{\mathrm{veq}} f/c_0$; `size_parameter_limit=10` caps the backlog.
dpl / dpl_{eff}	dipoles per (vacuum) wavelength — DDA resolution. `dpl_eff` is what the fixed shape file actually yields at the requested λ. ADDA's auto heuristic lands at $dpl_{\mathrm{eff}} \approx 18$ at production x.

Term	Meaning in this project
resolution class / fmin	each physical size exists as several shape files tagged `frequency_min ∈ {192, 337, 457, 671, 887} GHz`; the class decides `N_dipoles` (§1.2).
β grid	tilt of the particle symmetry axis from vertical — the ARO *output* coordinate (19 values in the SSDB grid). Not a solve axis.
incidence angle θ_{inc}	direction of the incoming plane wave relative to the *laboratory* z-axis — second ARO output coordinate (19 values). Not a solve axis either: ADDA solves per *icosphere vertex* (§1.5.2).
`-eps` (ADDA)	iterative-solver stop criterion: residual reduced by $10^{-\mathrm{eps}}$. `eps=3` production; noise floor §1.1.
`eps_sphexp` =0.005	per-vertex spherical-harmonic truncation error of the Mueller grid, relative to RMS of Mueller element M_{11} (§1.6).
`delta` =0.01	convergence tolerance of the icosphere orientation refinement loop (relative change of the averaged extinction matrices, §1.5.4).
`limit_sph` =4	max icosphere refinement level (levels: 12/42/162/642/2562 vertices; minimum 3 passes).
shtns	fast spherical-harmonics transform library (orthonormal, no Condon–Shortley phase here).
Matzler 2006	dielectric function of ice used for all ice habits: `acf.eps_ice_matzler06(f, T)`.
SSDB	ARTS Microwave Single Scattering Properties Database; our export format is `beta_0.9` (zenodo 1175572 lineage — Eriksson et al. 2018; ARO extension: Brath et al. 2020).
habit IDs	particle shape definitions from the ARTS habit catalogue: ID9 = Hong/MoCOLAG plate, ID7 = Hong column, ID20 = hexagonal plate aggregate, ID28 = Icon snowflake, ID3 = Liu sector snowflake.
gloden / baseline	frozen ADDA outputs in `baselines/` used for regression (§4.1).

0.3 Status at the time of writing

- **Done (computed + processed):** ARO for `HexPlaAggCrystal_Id20`, `HongPlate_Id9` (results on HPC, currently not accessible), plus analytic/voxel verification cases.
- **Backlog ready** (configs verified, jobs counted, not yet run): ARO `HongPlate_Id9` full 35-channel grid (5235 jobs), `HongColumn_Id7` (1524), `IconSnow_Id28` (1276), `LiuSectorSnowflake_Id3` (1252); TRO `HongPlate_Id9_TOR` (4608 jobs). Estimated ≈ 2700 single-core CPU-h total (see §4.4).
- **Production ADDA binary: 1.4.0** (locally built; equivalence-gated against 1.2 and 1.5.0-alpha3, §4.2). Cluster branch configs still default to 1.2 until an MPI 1.4 build exists there — override with `ADDA_BINARY` / `ADDA_VERSION` (§3.5).
- Before touching **any** pipeline code, read `plans/DDA–modernization–report.md` (§1.5 breakages, §1.6 latent-bug catalogue, Phase 2–5 sections list what is deferred).

Part 1 — Theory

1.1 The Discrete Dipole Approximation (DDA)

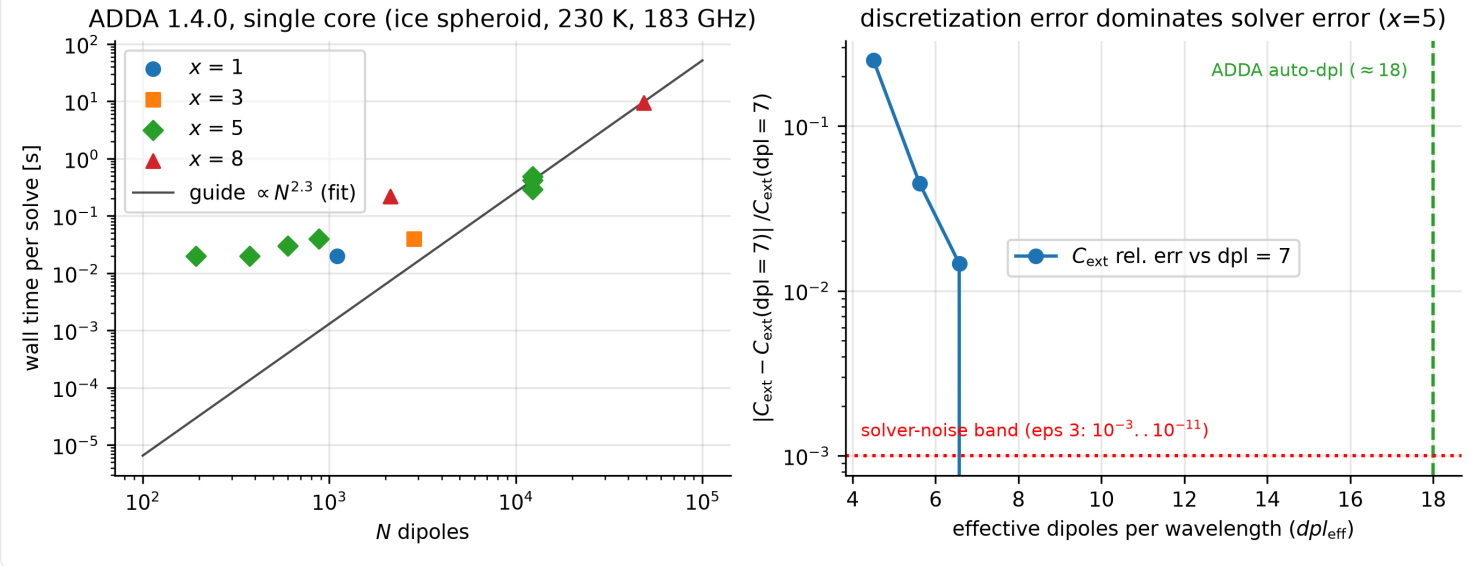
The DDA represents a particle of arbitrary shape as a finite array of **N polarizable dipoles** sitting on a cubic lattice with spacing *l*. Each dipole acquires a dipole moment $\mathbf{P}_i = \boldsymbol{\alpha}_i \, \mathbf{E}_{\mathrm{tot}}(\mathbf{r}_i)$, and the total field is the incident field plus the field scattered by *all* dipoles — which yields a dense linear system

$$\mathbf{E}_{\mathrm{tot}}(\mathbf{r}_i) \;=\; \mathbf{E}_{\mathrm{inc}}(\mathbf{r}_i) \;+\; \sum_{j \neq i} \mathbf{A}_{ij} \, \mathbf{P}_j, \qquad i = 1, \ldots, N$$

solved iteratively (ADDA default: GMRES stabilized with the Lanczos-type solver family) rather than by direct factorization. The particle shape is encoded simply by *which* lattice sites carry dipoles — this is why arbitrary habits (aggregates, snowflakes, columns) are tractable, and why the whole pipeline can be driven purely from `*.adda` position files.

What the maintainer must know about ADDA's numerics (all measured in `analysis/phase4b_adda_benchmark.py`, data in `analysis/phase4b_bench_results.json`, figure below):

Fact	Consequence for this project
`-eps <n>` = iterative solver stops at residual 10^{-n}. It controls **solver** error only.	`eps=3` production: own-solver scatter is $\lesssim 10^{-11}$ for small jobs but $\sim 8 \cdot 10^{-4}$ at $x = 5$ — so **identical ADDA re-runs are not bit-reproducible**; every regression gate must be noise-based, never bit-level.
Error budget is **discretization-dominated**: at $x = 5$ going dpl_{eff} from 4.5 to 7.5 moves Cext by $2.5 \cdot 10^{-1}$ → ref-value, while $\varepsilon_{\text{solver}}$ 4→2 moves it only $\sim 1.5 \cdot 10^{-3}$.	Tuning `-eps` below 3 buys nothing; resolution (the shape file class) is the lever (§1.2).
ADDA's auto dpl heuristic lands at $dpl_{\text{eff}} \approx 18$ at production $x \in (1, 8)$.	The fixed per-file dipole counts are deliberately over-resolved; this is a *design choice* (one geometry file reused across all frequencies of its class).
Cost per solve: $N = 12\,312$ **dipoles → 0.4 s,** $N = 48\,597$ **→ 9.5 s** (single core, ADDA 1.4.0, macOS). Scaling $\propto N^{2.3}$.	Backlog cost model §4.4; also why solves are embarrassingly process-level parallel (run N single-core ADDA calls per job, §3.6).
Cext/Cabs from a DDA run are only as good as the shape discretization; mirrors/rotations of the *voxel set* are exact lattice operations (§1.7).	Symmetry flags must be verified on the actual voxel set, not on the nominal habit.



Left: wall time vs dipole count per solve (single core, oblate spheroid 1.0/0.5, Matzler ice at 230 K, 183 GHz). Right: discretization error at x=5 — dpl_{eff} dominates; the auto heuristic (≈18) sits far to the right of the eps-noise band.

Key references: Yurkin & Hoekstra 2011, JQSRT 112, 2234–2247; ADDA manual `adda/doc/manual.pdf` and `adda_1.2/doc/manual.pdf` .

1.2 Particle shape files and the resolution scheme

Shape model and file contract

Every geometry is a pair of sibling files in `adda_shape_files/<habit>/` :

```
Dmax00013um_Dveq00010um_Mass4.79983e-13kg_Freq0192.000GHz.adda      ← dipole positions (x y z, one per line, la
Dmax00013um_Dveq00010um_Mass4.79983e-13kg_Freq0192.000GHz.meta.mat  ← MATLAB metadata struct 'meta'
```

The `.meta.mat` (read with `acf.loadmat`) drives *all* size/frequency bookkeeping. Fields actually consumed by the pipeline: `diameter_vol_eq` [m], `diameter_max` [m], `frequency_min` [Hz], `N_dipoles` , `mass` [kg], `lambda_dpl` (=λ at fmin, m — used to fill the SSDB `dpl` field), plus provenance (`habit_id` , `source` , `medium` , …). **A `.adda` without its `.meta.mat` sibling is invisible to the pipeline** (the configs glob `*.mat` for the size tables).

The model is **cubes on a cubic lattice**. Mirrors map the lattice onto itself — which is exactly what the symmetry-flag tests of §1.7 exploit. Rotations by non-multiples of 90° never do, so a voxelized hexagon is *never* an exactly C6-symmetric model; ideal-shape C6 exists sub-voxel only, and exploiting it would cost a boundary-layer discretization error. This is why rotation symmetries (`sym_Zobj`) are parked while mirror symmetries are exploited.

The 5-resolution scheme

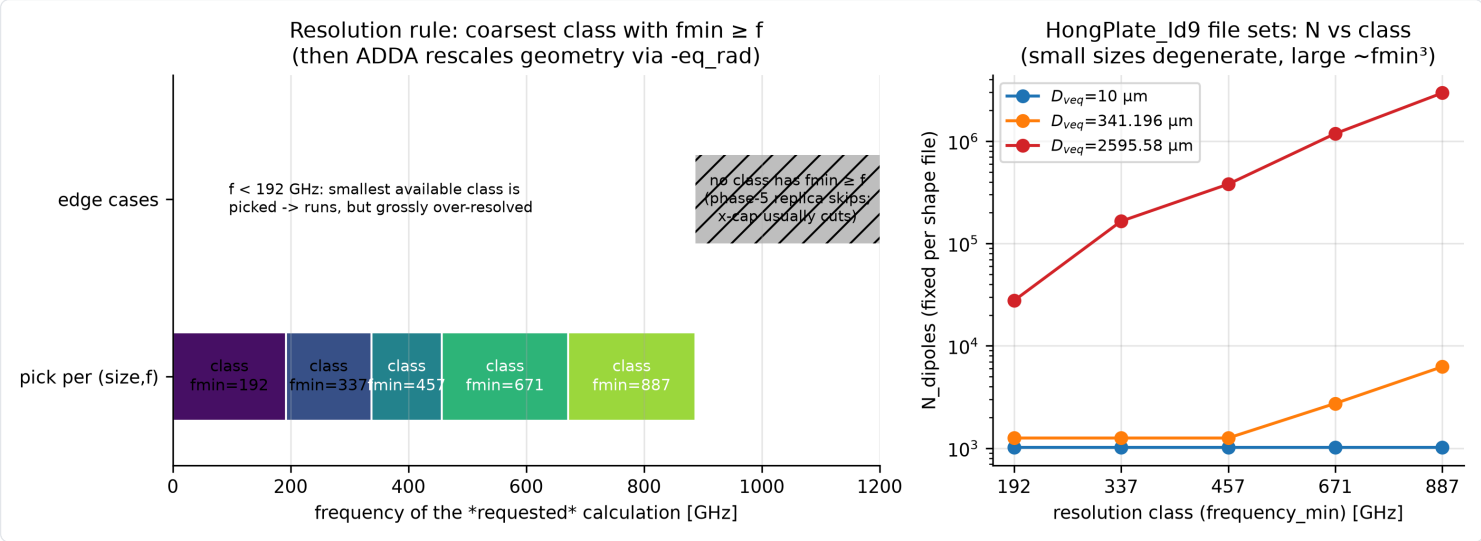
Each physical size ships as several shape files tagged with `frequency_min`:

- classes: **{192, 337, 457, 671, 887} GHz**;
- `N_dipoles` is **fixed per file** — ADDA is *never* asked to resample; it rescales the geometry (`-eq_rad`) so the same dipole array represents the particle at any λ;
- the effective resolution at frequency f is therefore $dpl_{\text{eff}} = dpl_{\text{design}} \cdot (f_{\min}/f)$ — i.e. a file only gets *finer* than designed when used below its fmin.

Two regimes exist in practice (measured on `HongPlate_Id9`, fig. right panel):

particle	classes	N_dipoles across classes
small (10 μm)	5	**identical** (1015 in all five — degenerate copies: even the coarsest lattice already exceeds the needed dpl)
mid (341 μm)	5	1258 for classes 192/337/457, then 2740, 6255
large (2596 μm)	5	27 644$\to **165072**\to **379780**\to **1184589**\to $2 956 210 ($\approx f_{\min}^3$)

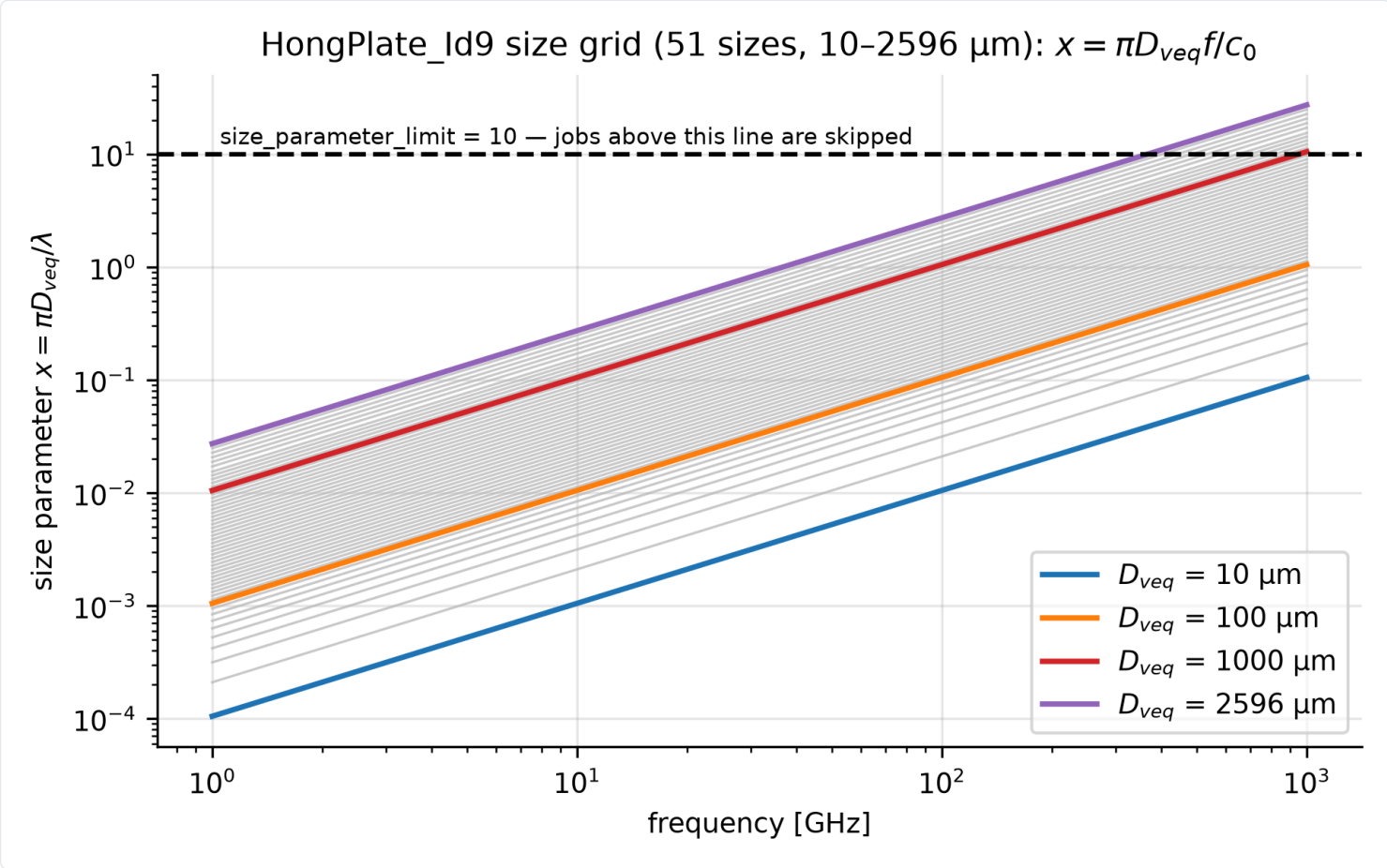
Also note: **class availability can be per-size incomplete** (e.g. HexPlaAgg 197 μm has only the 671 and 887 classes), so never assume five files per size.



The selection rule (as implemented in `adda_main_control.py`)

For each (size, f): among the files of that physical size, the code picks the **coarsest class with fmin ≥ f** (`df = fmin−f; df[df<0]=inf; argmin`); every *other* file of that size prints "wrong particle realisation" and skips. Consequences, precisely:

- f inside a class interval $(f_{\min,k-1},\ f_{\min,k}]$ → class k computes it ($dpl_{\text{eff}} \geq$ design value — always on the safe/over-resolved side);
- **f below all classes** (e.g. the 1–175 GHz entries of the frequency list) → the *smallest available class* is picked and the job **does run**, grossly over-resolved;
- **f above all classes** → with `df` all-∞ the raw control code would accidentally pick the first file in folder order; the analysis replicas (`phase5_backlog_inventory.py`) additionally guard `if not np.isfinite(df).any(): continue`. In production the x-cap removes these combinations anyway (HongPlate's 886 GHz < 887 class).
- The size cap: $x = \pi D_{\text{veq}} f/c_0 >$ `size_parameter_limit` $(= 10)$ → skip (146 of HongPlate's 8925 (size,f) combos; the *majority* of combos are killed by resolution bookkeeping, not by x — 7701 by the fmin rule; report §1.7).



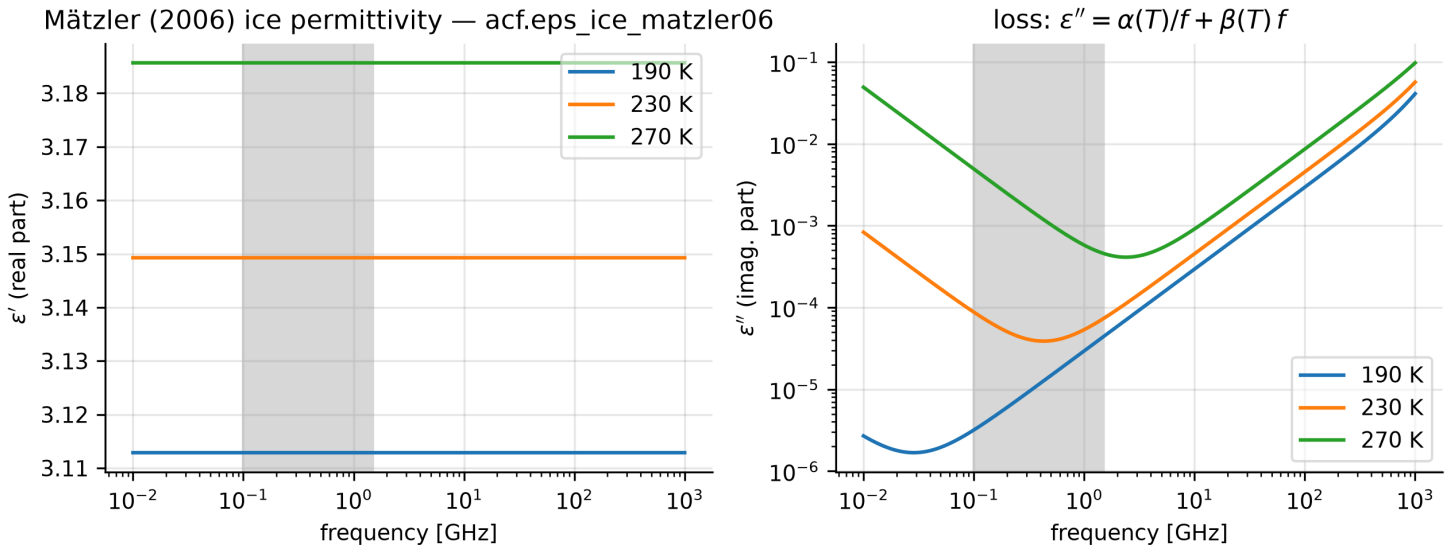
HongPlate_Id9 size grid (51 sizes, 10–2596 µm): the x-cap cuts the top-right corner.

1.3 Dielectric function of ice (Matzler 2006)

All ice habits use the Mätzler (2006) parametrization, implemented in `acf.eps_ice_matzler06(f, TK)` (`adda_control_functions.py:1068–1103`):

$$\varepsilon(f,T) \;=\; 3.1884 + 9.1\cdot 10^{-4}\,(T-273) \;+\; i\left[\frac{\alpha(T)}{f} + \beta(T)\,f\right], \qquad f \text{ in GHz},\ T \text{ in K}$$

Valid for 0.01–3000 GHz, 20–273.15 K (raises otherwise). **It returns the complex permittivity, not the refractive index** — callers compute $m = \sqrt{\varepsilon}$ and hand `-m m_re m_im` to ADDA. Note the absorption is strongly frequency-dependent (loss $\propto 1/f$ at low f, $\propto f$ at high f) and strongly temperature-dependent: the imaginary part at 190 K vs 270 K differs by orders of magnitude — hence the three production temperatures {190, 230, 270} K are *not* a formality; each (T, f, size) is a separate ADDA job.



Grey band: the sub-mm window region. The loss tangent at mm/sub-mm wavelengths is what makes scattering by cloud ice a microwave-radiometry issue at all.

1.4 From ADDA output to Mueller and extinction matrices

Each ADDA solve produces two parsed artifacts (parsing in `acf.adda_call`, `adda_control_functions.py:198–236` — **positional, column indices hardcoded**; re-verify when switching ADDA versions, report §1.3):

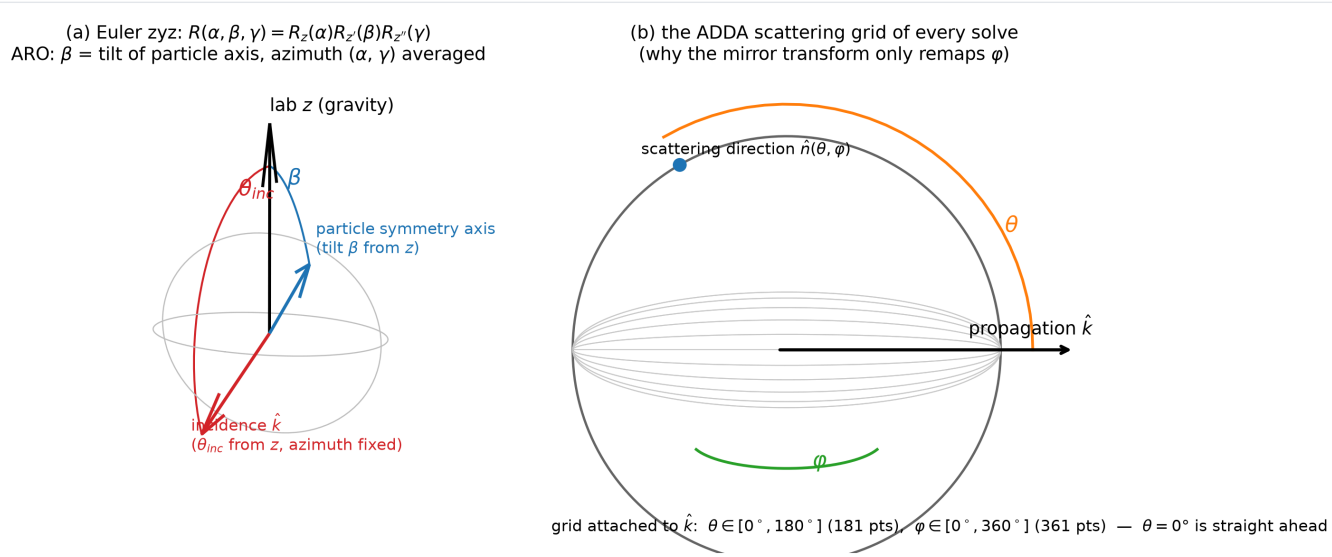
1. **`ampl_scatgrid`** — the complex *amplitude matrix* S(f̂) on the scattering grid. Row 0 = forward direction (θ=0). Column layout (real/imag pairs) is **not** in textbook order:

cols	quantity
0, 1	s_{22}
2, 3	s_{11}
4, 5	s_{12}
6, 7	s_{21}

From the forward amplitudes the **extinction matrix** (4×4, units $\mu\mathrm{m}^2$) is built via the Mishchenko (2002) optical theorem construction — amplitudes scaled by $\lambda/(-2\pi i)$, then $K\cdot = \lambda$ (`acf.extinction_matrix`, :40–80). *The ARO pipeline's C_ext comes entirely from this grid column — which is why `-Csca` and `-alldir_inp` were removed from all ARO configs (Phase 3): nothing ever parsed them.*

2. **`mueller_scatgrid`** — the 16 Mueller-matrix elements M_{11}…M44 (row-major) on the scattering grid from `scat_params.dat`: **181 θ (0–180°) × 361 φ (0–360°), 1° resolution**; col 0 = θ, col 1 = φ (degrees). This is the *angular scattering signature of one fixed particle orientation* — the object that §1.6 expands in spherical harmonics. The θ/φ here live on a grid **attached to the propagation direction** — a crucial fact for the mirror transforms of §1.7 (θ is never remapped).

The two output flags `-scat_matr both` request amplitude + Mueller; `-store_scat_grid` + `-scat_grid_inp scat_params.dat` request the grid itself. The geometry of *both* angular coordinate systems — the lab-frame incidence/tilt angles and the propagation-attached scattering grid — is shown below:



1.5 Orientation averaging — the two strategies

1.5.1 Why averaging, and in which coordinates

A real cloud contains particles in many orientations. The observed scattering is the orientation average of the single-particle Mueller matrix. Euler angles are taken in ADDA's **zyz convention** (α about lab-z, β about the line of nodes, γ about the particle's own z), i.e.

$$R(\alpha,\beta,\gamma) \;=\; R_z(\alpha)\, R_{z'}(\beta)\, R_{z''}(\gamma), \qquad \text{ADDA flag } \{\texttt{-orient}\}\ \alpha\ \beta\ \gamma\ [\text{deg}]$$

> **Source methodology (ARO):** the TRO/ARO classification, the zyz Euler convention, the α/γ-averaging at fixed tilt β, and the resulting ARO dependence on (β, θ_{inc}, scattering angles) follow **Brath et al. (2020, AMT 13, 2309–2333)** — the foundation paper of this project's ARO approach (see References). Their hexagonal-plate / plate-aggregate ARO database itself builds on the shape and frequency/temperature setup of **Eriksson et al. (2018, ESSD 10, 1301–1326)**, which is also where the habit IDs used in this repo (`HongPlate_Id9` , `HongColumn_Id7` , …) originate.

	ARO	TRO
free orientation parameters	tilt β of the particle axis; azimuth (α, γ) averaged	all of α, β, γ
physically motivated state	axis horizontal: $\beta = 90°$ (gravity-brained crystals)	freely tumbling particle
what is produced	the full grid $\beta \in [0°, 180°]$ (19 values) — users weight the tilt distribution downstream	orientation-independent result (integrated inside ADDA)
weighting	uniform in α, γ at each β	uniform over the rotation group

The SSDB represents the ARO state on a 2-D output grid **(β, θ_{inc})**: β = tilt of the particle axis from vertical (0–180°, 19 output values), θ_{inc} = angle between the incoming plane wave and the lab-z axis (0–180°, 19 output values, azimuth of incidence fixed to 0 — the remaining azimuthal randomness is what makes it "azimuthally random"). The α and γ integrations are performed numerically (Gauss–Legendre) at every (β, θ_{inc}) node.

> **Critical design fact (Phase-4b, report §4b.1):** the β and θ_{inc} grids are **SSDB output coordinates, not quadrature axes**. The number of ADDA *solves* is set solely by the icosphere orientation-vertex grid (§1.5.2) — changing `number_of_betas` or the incidence-angle resolution does not save or cost ADDA solves. And the tempting “half-range reuse” symmetry (solve β<90° and mirror to β>90°) is **provably wrong** (measured discrepancies $2.3 \cdot 10^{-3}$…$7.2 \cdot 10^{-2}$ vs 10^{-11} solver noise, `analysis/phase4b_halfrange_check.py`).

1.5.2 ARO sampling: the icosphere vertex grid

One ARO job (one T, f, size) samples particle orientations as the **vertices of a refined icosahedron** on the unit sphere. A vertex *is* the incidence direction: ADDA is called with `-prop vx vy vz` (fixed particle attitude `-orient 0 0 0`, beam coming from the vertex direction). Vertex counts per refinement level L:

$$V(L) = 10 \cdot 4^L + 2 \;: \quad 12,\ 42,\ 162,\ 642,\ 2562, \ldots \qquad (\text{triangles } 20 \cdot 4^L)$$

per vertex: ADDA solve → parse Mueller grid → SH-expansion (§1.6) → per-vertex NetCDF cache. The full α/β/γ machinery then reconstructs the orientation average by rotating this vertex-resolved data set (`ico.rotate_vertices` + barycentric interpolation on the rotated icosphere + Stokes polarization-basis rotation `aoa.pol_changer`).

1.5.3 ARO averaging quadrature

Inside `adda_sub_control_spharmonics.py` (:629–814) the **extinction matrix** is averaged at each $(\beta, \theta_{\text{inc}})$ node with Gauss–Legendre (GL) quadrature in α and γ:

$$n_{\text{GL}} = 4 \cdot 2^L \qquad \text{GL nodes mapped to } [-180°, +180°] \text{ in } \alpha \text{ and } \gamma,$$

for every β (from `number_of_betas`, 5) and θ_{inc} (`incidence_angles`, 10 values): rotate the vertex set by (α, β, γ) → barycentric interpolation of the extinction matrix at the incidence direction → Stokes polarization-basis rotation → GL-weighted sum over γ, then over α.

The **Mueller** SH-coefficient averaging happens later, in post-processing (`aoa.rawdata2average`, joblib-parallel, output re-gridded to the 19×19 SSDB grid).

1.5.4 Convergence of the orientation sampling

The icosphere level L is refined until the averaged extinction matrices stop changing:

$$\varepsilon_{\text{rel}} \;=\; \frac{\left\| \overline{K}^{(L)} - \overline{K}^{(L-1)} \right\|_2}{\left\| \overline{K}^{(L-1)} \right\|_2} \;\le\; \delta\,(= 0.01), \qquad \overline{K} \in \mathbb{R}^{n_\beta \times n_\theta \times 4 \times 4}$$

at least 3 passes ($L = 0, 1, 2$), at most `limit_sph` $(= 4)$.

Note the two *different* accuracy controls and what each drives:

knob	drives	tolerance
`delta`	icosphere **orientation** refinement (# ADDA solves)	0.01 on averaged extinction
`eps_sphexp`	per-vertex **spherical-harmonic truncation** of the Mueller grid	0.005 rel. to M_{11} RMS
`-eps`	ADDA internal solver residual	10^{-3} (noise floor, §1.1)

1.5.5 TRO: two strategies, one decision

strategy	mechanism	yields	status
ADDA built-in `-orient avg <avg_params.dat>`	Romberg integration over α,β,γ *inside ADDA*; writes integrated `mueller` (2° θ-grid) + `CrossSec*` files	cross sections **and** orientation-averaged Mueller	**production for TRO** (Phase-5 decision, report §5.1)
own sampling + SH expansion (icosphere, as ARO)	same machinery as ARO but full Euler coverage	Mueller grids → SSDB	reserved for hypothetical TRO *grid* products; not needed for the backlog

Hard constraint discovered in Phase 3: **ADDA 1.4/1.5 reject `-orient avg` combined with `-scat_grid`, `-Csca` or `-alldir`** — built-in averaging never produces *per-angle* scattering grids (the integrated mueller is a separate product). The avg file (zyz, per-angle Romberg control `Jmin/Jmax/eps/equiv/periodic`) for the production habit — `adda_aux_input/HongPlate_Id9_TOR_sizegrid/avg_params.dat`:

Euler angle	range	$J_{\min}/J_{\max}$	tol.	equiv	periodic	note
α	$0 \ldots 360°$	2 / 5	0	true	true	periodic → trapezoid + Richardson
β	$0 \ldots 90°$	2 / 4	10^{-6}	false	false	**XY-mirror fold** (§1.7!)
γ	$0 \ldots 360°$	2 / 4	10^{-6}	true	true	

Folding β to 0–90° is *only legitimate because the voxel set is XY-mirror closed (closure 1.000 on the TOR grid)*. It was **gated** against the full-range file (`avg_params_fullrange.dat`): CrossSec rel-diff $\leq 1.35 \cdot 10^{-5}$ (= the Romberg tolerance itself), Mueller $\leq 10^{-13}$ → the fold is exact to integration accuracy (`analysis/phase5_tro_beta_gate.py` , PASS, report §5.3).

1.6 Spherical-harmonic expansion over the scattering angles

Every ADDA solve returns the Mueller grid $M_1, \ldots, M_{16}(\theta, \varphi)$ on the 181×361 scattering grid. Rather than storing 130k floats ×16 per vertex, the pipeline stores a **spectral representation** per vertex (`spherical_harmonics_for_adda.py`):

- transform: `shtns.sht(ntrunc, ntrunc, 1, sht_orthonormal | SHT_NO_CS_PHASE)` — orthonormal, no Condon–Shortley phase; input field interpolated from the θ-grid onto Gaussian latitudes;
- **error measure** — the L^2 sphere norm of the analysis→synthesis round-trip residual of the truncated expansion:

$$\delta[f] \;=\; \sqrt{\iint |f_{\mathrm{rec}}(\theta,\varphi) - f(\theta,\varphi)|^2 \,\sin\theta \,\mathrm{d}\theta \,\mathrm{d}\varphi}\,,$$

per Mueller element the absolute target is $\varepsilon_m = \mathrm{RMS}(M_{11}) \cdot$ `eps_sphexp` ; the returned relative error is $\delta[M_m] / \mathrm{RMS}(M_{11})$; - if the round-trip passes at full truncation, a **binary search** finds the *smallest* `ntrunc` still meeting `eps_m` — coefficient counts are per-vertex and adaptive; - coefficients are stored as **complex64** (this sets the floor of export precision; `map_sphexp` / `reshape_sphexp_2_common_exp` later remap every (β, θ_{inc}) node's coefficient set into one common shtns layout).

Why expand over scattering angles at all? Because the orientation average at a rotated orientation is then a **Wigner rotation of coefficients**, and phase-matrix products needed by radiative transfer stay fast; the post-processing additionally *re-expands* the averaged phase matrix on the fly to `eps_phase=0.001` on the 19×19 grid (§3.3 step 2).

1.7 Mirror-symmetry exploitation (Phase 4)

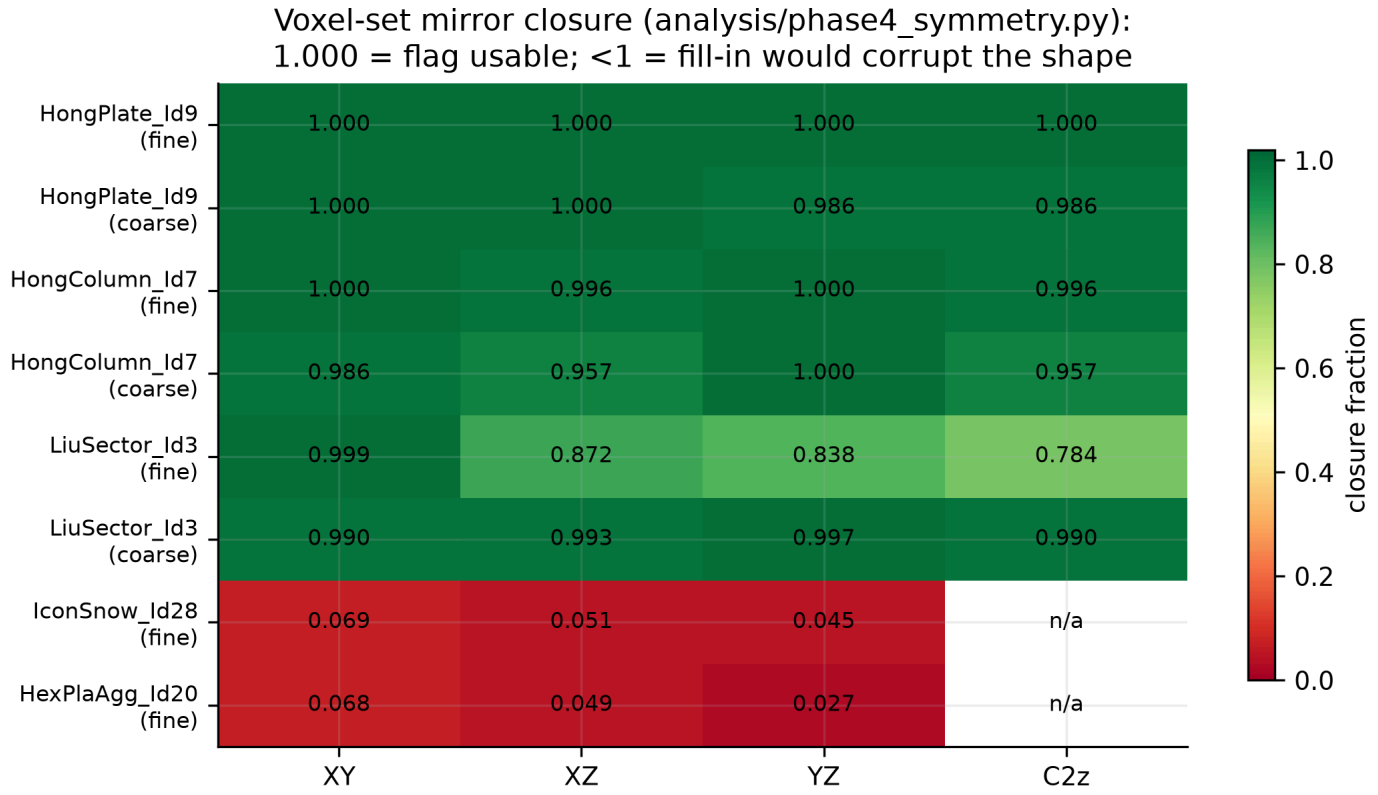
If the **voxel set** is invariant under reflection at a coordinate plane of the particle frame, the Mueller grid of the mirrored orientation is obtained from the solved one by an exact index/sign transform — the ADDA solve is skipped and the vertex is *filled*.

Verified transform (`acf.MIRROR_*` , `adda_control_functions.py:1252–1289` ; derived empirically, `analysis/phase4_grid_symmetry.py` , residual $\leq 3 \cdot 10^{-11}$ at `-eps 3`):

mirror (plane)	axis flipped	scattering-grid remap	Mueller element signs
XY ($z \to -z$)	z	$\varphi \mapsto 180° - \varphi$	for an odd number of mirrors: element-wise sign by $\mathbf{d}\mathbf{d}^{\top}$ with $\mathbf{d} = (1, 1, -1, -1)$ (outer product on the Stokes row/col pairs)
XZ ($y \to -y$)	y	$\varphi \mapsto -\varphi$	
YZ ($x \to -x$)	x	$\varphi \mapsto -\varphi$	(even number of composed mirrors → identity sign)

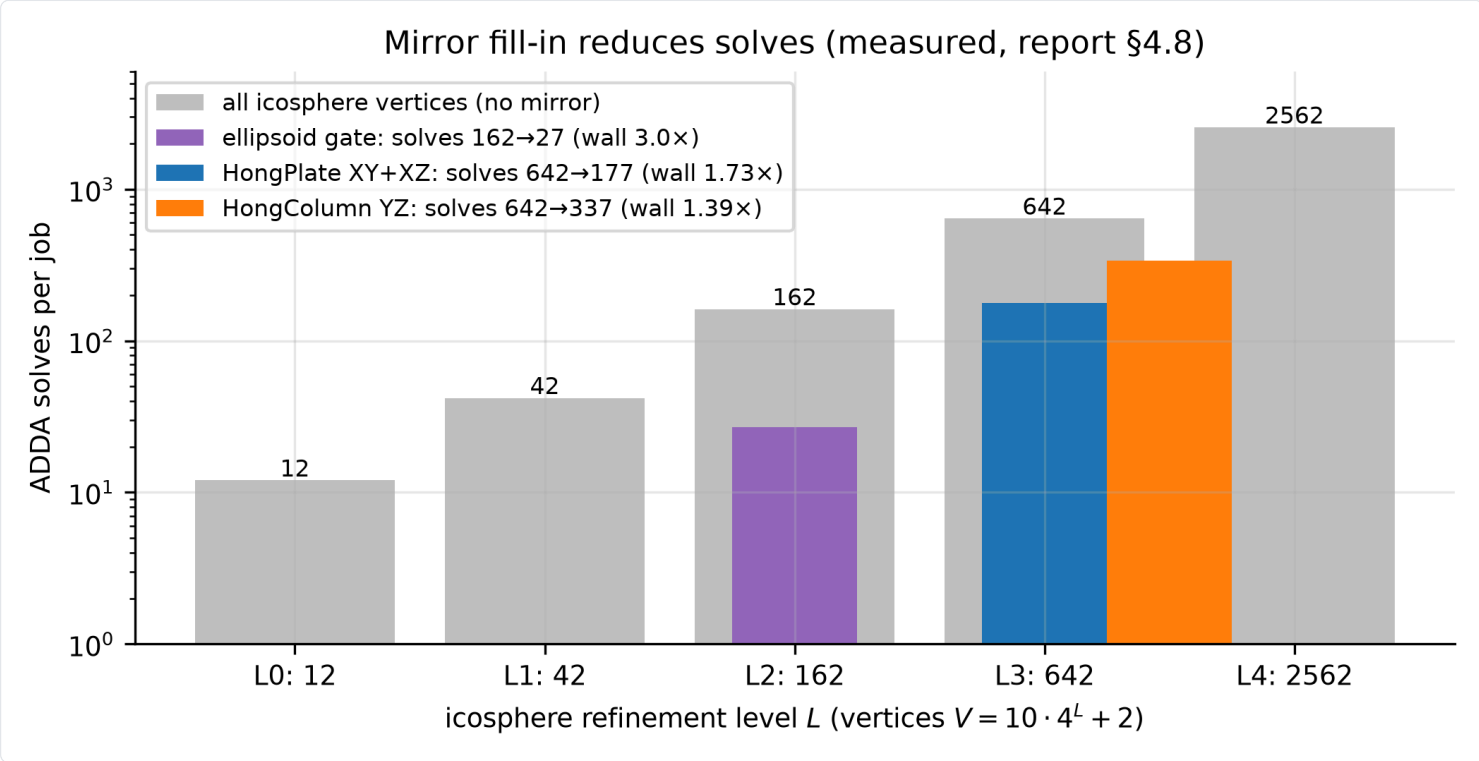
θ is never remapped — `mueller_scatgrid` lives on a propagation-*attached* grid; only φ moves. Fill-in mechanics in the sub-control (:265–322, :414–419): vertices on the negative side of an enabled plane are solved only if no fill exists yet (`_runfirst` ordering guarantees every mirrorable vertex is filled before it is reached; a missing fill can never silently produce NaN); the image vertex is found by nearest-vertex lookup (warns if snapping exceeds 10^{-3} rad); the filled grid is re-expanded in SH and cached like a solved vertex.

Precondition — voxel closure. A flag is legal only if the *voxel set itself* is mirror-closed, and closure is **resolution-dependent** — measured by `analysis/phase4_symmetry.py` (figure): a fill across a non-closed mirror is a real **shape error** (HongPlate coarse-class YZ: 0.986 closure → $1.5 \cdot 10^{-4}$ Mueller field error vs $4.5 \cdot 10^{-12}$ when closed), not solver noise. Hence flags must be set from the **coarsest resolution class** of the size grid.



Production flags (report §4.7): **HongPlate_Id9 + TOR grid: XY+XZ=true, YZ=false**; **HongColumn_Id7: YZ=true**; LiuSector/Icon/HexPlaAgg: none (aggregate/snowflake shapes break all mirrors). Measured effect (§4.8):

habit	solves	wall
ellipsoid gate (3 mirrors)	162 → **27** (+135 fills)	3.0×
HongPlate_Id9 (XY+XZ)	642 → **177** (+465 fills)	1.73×
HongColumn_Id7 (YZ)	642 → **337** (+305 fills)	1.39×



Wall gain is below the solve-count ratio at probe size because the per-*vertex* work (SH expansion + cache write) still happens for fills; at production $x \approx 10$ the ADDA solve dominates and the ratio closes. `sym_Zobj` (C_n rotation) is parked: exact C_{2z} itself is resolution-dependent (HongPlate: 1.000 fine / 0.986 coarse), and the cube-lattice model never realizes ideal C_6 exactly; XY+XZ fill-in already implies exact C_{2x} anyway.

1.8 What the SSDB export contains

Post-processing turns the per-job store into ARTS-SSDB NetCDF (`beta_0.9`) — the format of the published ARTS Microwave SSP Database, whose TRO generation is documented in **Eriksson et al. (2018, ESSD 10, 1301–1326)**; the ARO extension of that format is documented in **Brath et al. (2020, AMT 13, 2309–2333)**. One file per **(β, size, f, T)** under a nested tree:

```
<adda_results>/SSD/Azimuthally_random/ice/Aggregate/Pristine/HongPlate_Id9/
    AzimuthallyRandom_beta090deg/
        Dmax00019um_Dveq00017um_Mass1.086e-11kg/
            Freq166.900GHz_T270K.nc
```

Physically meaningful content per file (all spectral, units $\mathbf{m^2}$):

- `phaMat_data_real/imag` — phase-matrix spherical-harmonic coefficients, dims (16 Mueller elems × 1 az-inc × 19 za_inc × n_sph_coeffs);
- `extMat_data` — extinction-matrix elements (K11, K12, K33) on the (az, za_inc) grid;
- `absVec_data` — absorption vector (2 elems), computed as $C_{\text{ext}} - C_{\text{sca}}$ from the spectra, with a **negative-a1 rescale diagnostic** (§3.3);
- `ShapeData` scalars: diameters, mass, dpl, N_dipoles, refractive index (real/imag), plus provenance attrs (`refrIndex_model='Maetzler2006'` , habit id/name, source file);
- `CalculationData` attrs (provenance of the run): ADDA version, cluster, n_cores, `ADDA_eps` , `Refinement_delta` , `Mueller_sph_eps` , `Phase_sph_eps` , symmetry flags, shtns version.

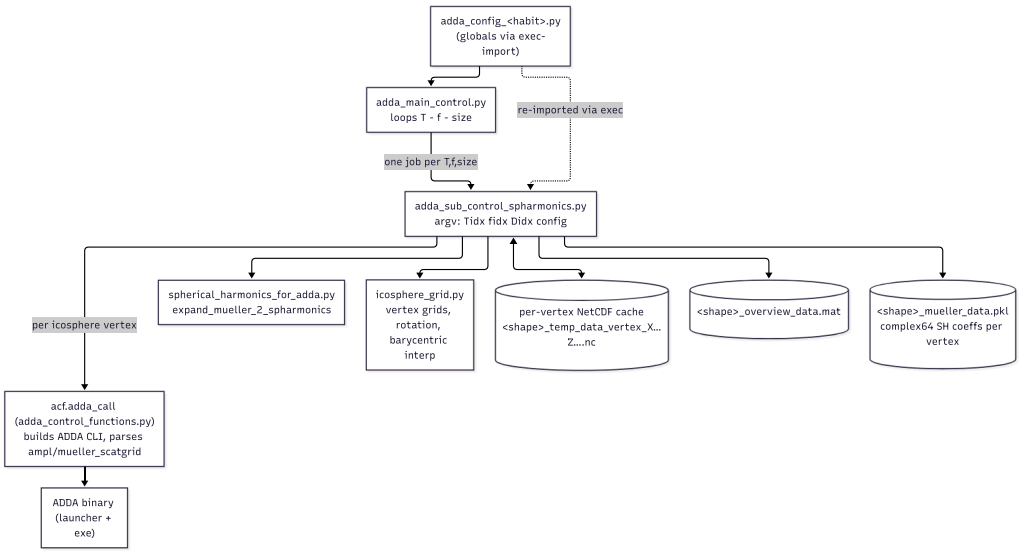
The derived quantities consumers usually want (C_ext, SSA, asymmetry parameter g) are **not** stored — they are computed downstream by the published-DB tooling (`arts-sspdb-interface/`). Full variable table: Appendix C.

Part 2 — Architecture & data flow

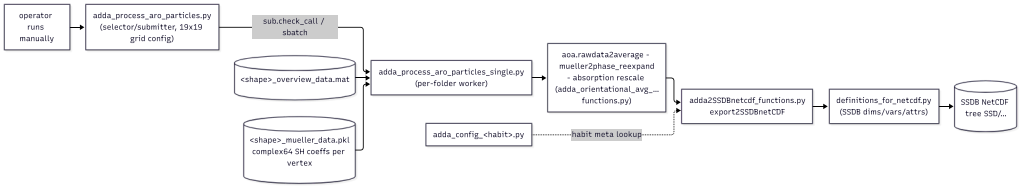
2.1 ARO call graph (production path)

Verified call graph (report §1.1, line numbers re-verified against current files):

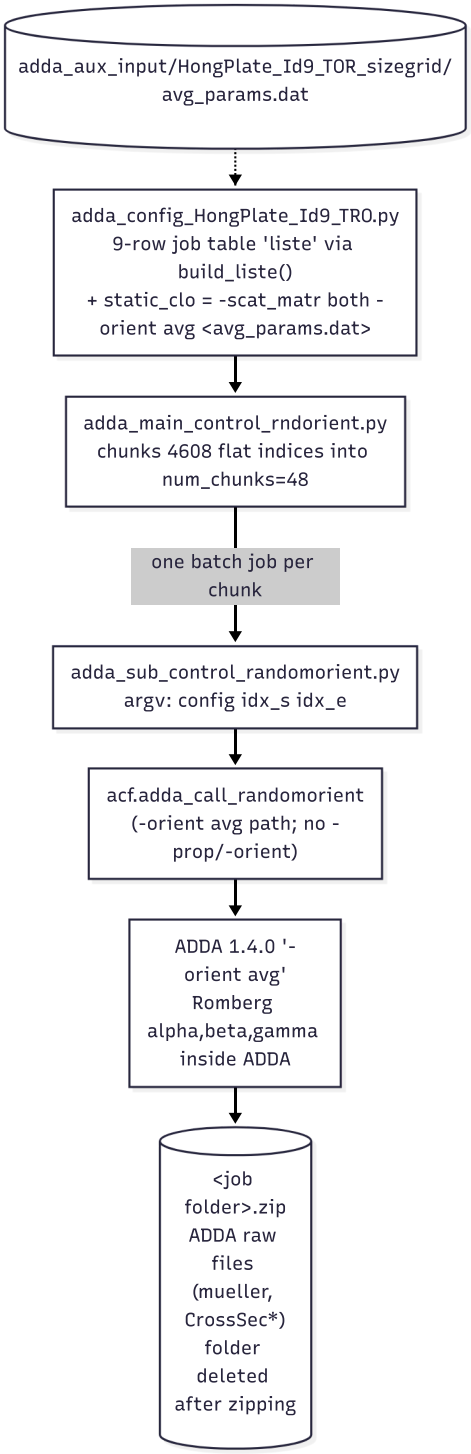
Solve phase (one job per T, f, size):



Post-processing phase (run later, per habit — see §3.3):

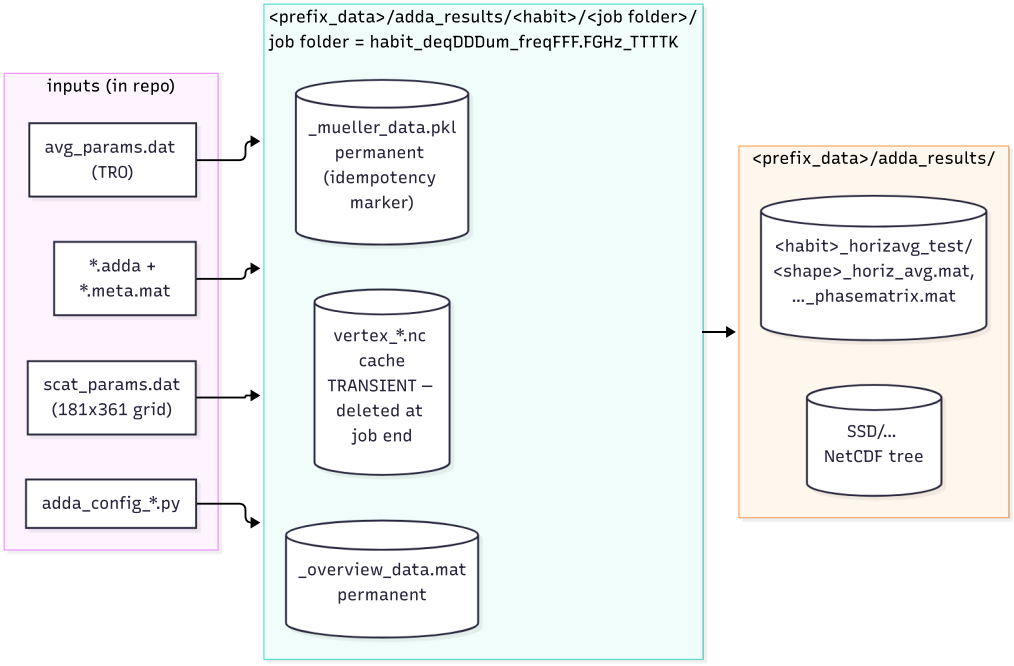


2.2 TRO call graph



Gotchas specific to TRO (all verified): the mistral batch branch of `adda_call_randomorient` raises `NotImplementedError` (never was implemented — now loud instead of a swallowed NameError); there is **no completion-skip logic** (every listed index is re-run); `liste` **row 8 must carry the temperature** (9-row format) or the folder gets named with the flat index as `T####K` (a loud warning guards this legacy path); `deq` in the folder name receives the equivalent **radius** (legacy naming).

2.3 Artifact data flow (what lives where, what is transient)



Idempotency layers (ARO): **(1)** job-level — main control skips a (T,f,size) if the folder contains exactly one `*mueller_data.{xml,pkl}` and `force_new=False`; **(2)** vertex-level NetCDF cache (survives a killed/restarted *job*, deleted only when the job completes); **(3)** within a refinement pass, vertices identical to the previous icosphere level reuse cached expansions without touching ADDA. TRO has **none** — plan reruns accordingly.

2.4 Module status table

module	role	status
`adda_main_control.py`, `adda_sub_control_spharmonics.py`	ARO driver + per-job solver	**active**
`adda_main_control_rndorient.py`, `adda_sub_control_randomorient.py`, `acf.adda_call_randomorient`	TRO driver (`-orient avg`)	**active** (Phase-3 repaired)
`adda_control_functions.py` (acf)	ADDA CLI wrapper, Matzler dielectric, Romberg helpers, mirror transforms, NetCDF cache, MATLAB reader	**active**
`spherical_harmonics_for_adda.py` (spha), `icosphere_grid.py` (ico)	SH machinery, orientation grids	**active**
`adda_orientational_avg_spharmonics_functions.py` (aoa)	post-processing averaging / phase-matrix re-expansion	**active**
`adda_process_aro_particles[.py/_single.py]`, `adda2SSDBnetcdf_functions.py`, `definitions_for_netcdf.py`	post-processing + SSDB export	**active** (export gated behind env `SSDB_EXPORT_NETCDF=1`)
`adda_sub_control_gridref.py`, `acf.romberg_alpha/gamma`, `adda_orientational_avg_spharmonics_for_testing.py`	old grid-refinement experiment	**deprecated — do not extend**
`posix_spawn.py`	fast process spawn (Linux ctypes; macOS subprocess branch)	active helper
`memory.py`, `sbatch_adda`	legacy helpers	dormant
`old_configs/`, `SSD_scripts_deprecated/`, `NetCDF_export_testscript*.py`	archives / one-off test scripts	**archive — never import**

Part 3 — Operations & runbooks

3.1 The config contract

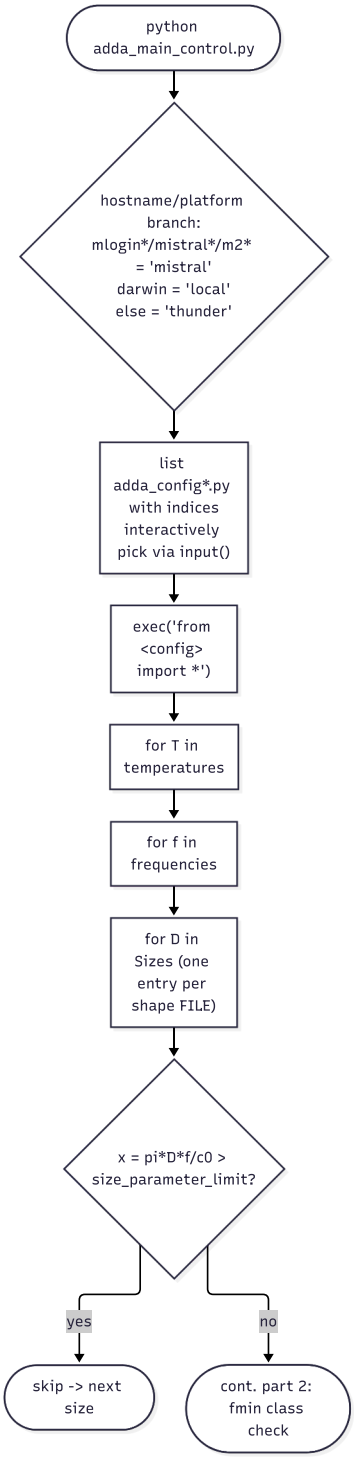
Every habit has one `adda_config_<habit>.py`; the control scripts inject it with `exec('from adda_config_X import *')` and read its globals. **Keep field names** when copying a config — the drivers `getattr`-probe them. The canonical reference is `adda_config_HongPlate_Id9.py`. Annotated field → consumer map:

config global	meaning	consumed by
`calc_type` (`'mistral'` / `'thunder'` / `'local'`)	host mode; **`local` = Phase-3 no-queue macOS branch** (repo-relative paths, direct `sub.check_call`)	both main controls, sub-controls, export provenance
`prefix`, `prefix_data`, `main_path`, `ramdrive`	storage roots (local: repo-relative; `ramdrive=None` → system temp)	sub-controls (job folders, ADDA `-dir`)
`adda_prefix`, `cluster_name`	ADDA checkout root; SSDB provenance string	export `CalculationData`
`adda_binary`, `adda_launcher`, `adda_version`	ADDA executable triple; env overrides `ADDA_BINARY` / `ADDA_VERSION` apply on top; sub-controls fall back to 1.2 defaults if a config lacks these	both sub-controls via `main_command = launcher + [binary]`
`shape_name`, `shape_foldername`, `shape_path`, `subpath`, `shapelist`, `shape_str`	habit + geometry file list (`shape_str=[]` ⇒ `-shape read <file>` is prepended)	main/sub controls, export meta lookup
`Sizes`, `Dmax`, `fmin` (arrays parallel to `.mat` files)	physical size grid + resolution classes (read from `*.meta.mat` with `acf.loadmat`)	job loops + selection rule
`frequencies` (Hz), `temperatures` (K)	the production channel set (35 channels 1–886 GHz × 3 T for HongPlate)	job loop; dielectric evaluation
`size_parameter_limit` (=10)	x-cap	main control skip
`number_of_betas` (=5), `incidence_angles` (10 vals)	ARO **output** coordinate resolution during solve-averaging (post-processing re-grids to 19×19)	sub-control averaging stage
`delta` (=0.01), `limit_sph` (=4)	icosphere orientation convergence / max level	sub-control refinement loop
`eps` (=3)	ADDA solver residual exponent	`acf.adda_call -eps`
`eps_sphexp` (=0.005)	per-vertex Mueller SH truncation target	`spha.expand_mueller_2_spharmonics`
`sym_XYobj`, `sym_XZobj`, `sym_YZobj`	**mirror fill-in flags — must be set from the coarsest resolution class** (§1.7, `analysis/phase4_symmetry.py`)	sub-control skip/fill logic; export provenance
`alpha_0/beta_0/gamma_0` (0/0/0)	base particle attitude handed to `-orient`	sub-control
`force_new` (False)	recompute even if completed-marker exists	main control idempotency
`ZIPPING` (False)	keep per-vertex ADDA output in zips (auto-deleted if False)	`acf.adda_call`
`scat_param_path`, `static_clo`	ADDA static flags: `-store_scat_grid -scat_grid_inp <scat_params.dat> -scat_matr both` (`-Csca` / `-alldir_inp` removed — never parsed)	`acf.adda_call`
TRO only: `liste` (9 rows!), `num_chunks`, `batch_time`, `avg_param_path`	flat job table (rows: shape, **eq radius m**, eps, m_re, m_im, **λ** m, shape file, batch time, **T K**) + chunking + avg file	rndorient pair
HPC: `ncores`, `Partition`	Slurm sizing	sbatch wrappers

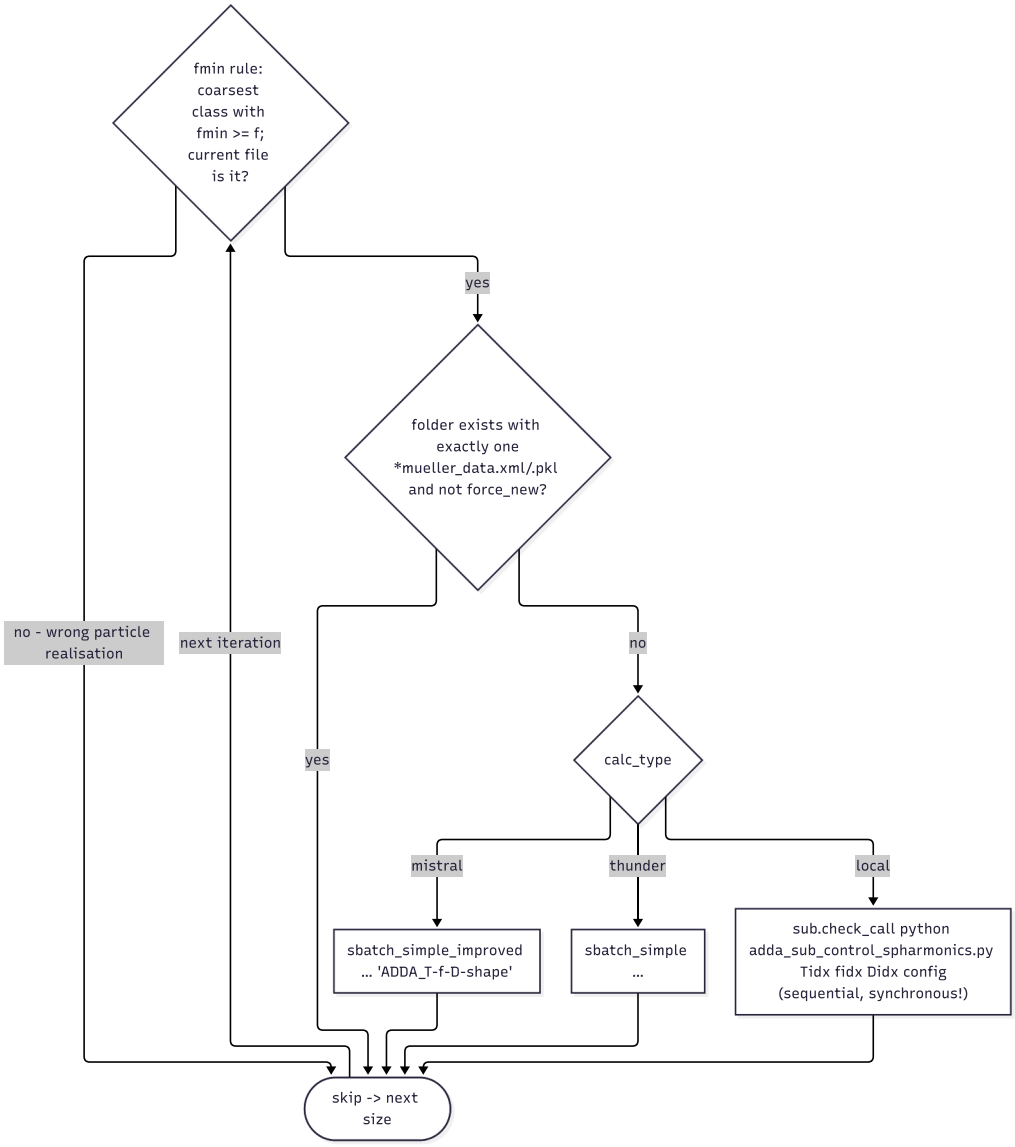
3.2 ARO job flowcharts

Driver: `adda_main_control.py`

Part 1 — startup and the nested loops:



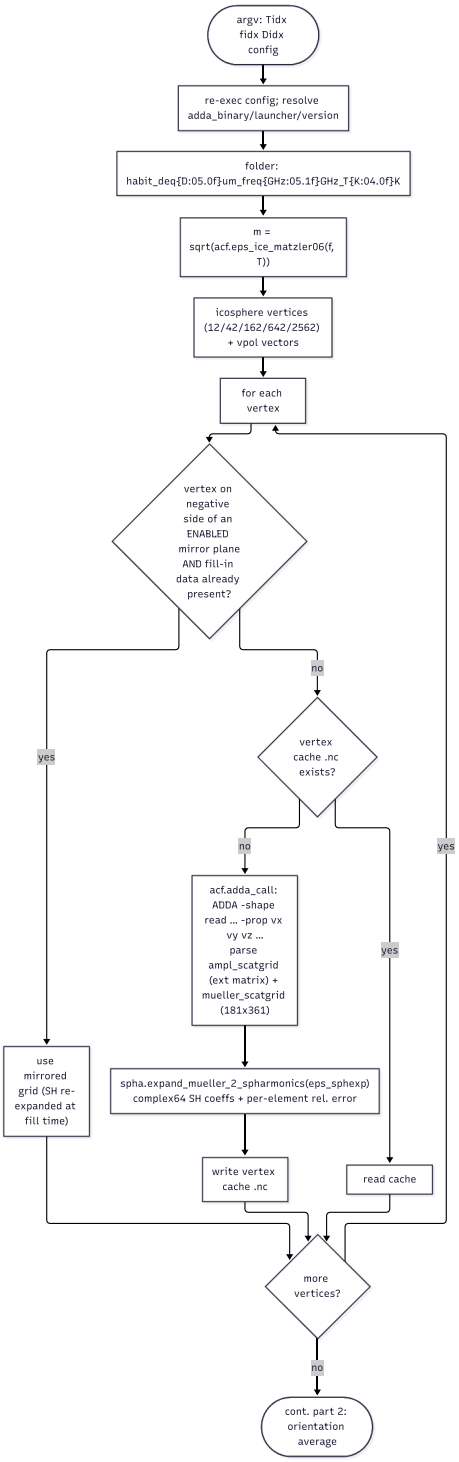
Part 2 — resolution-class selection, idempotency check, dispatch:



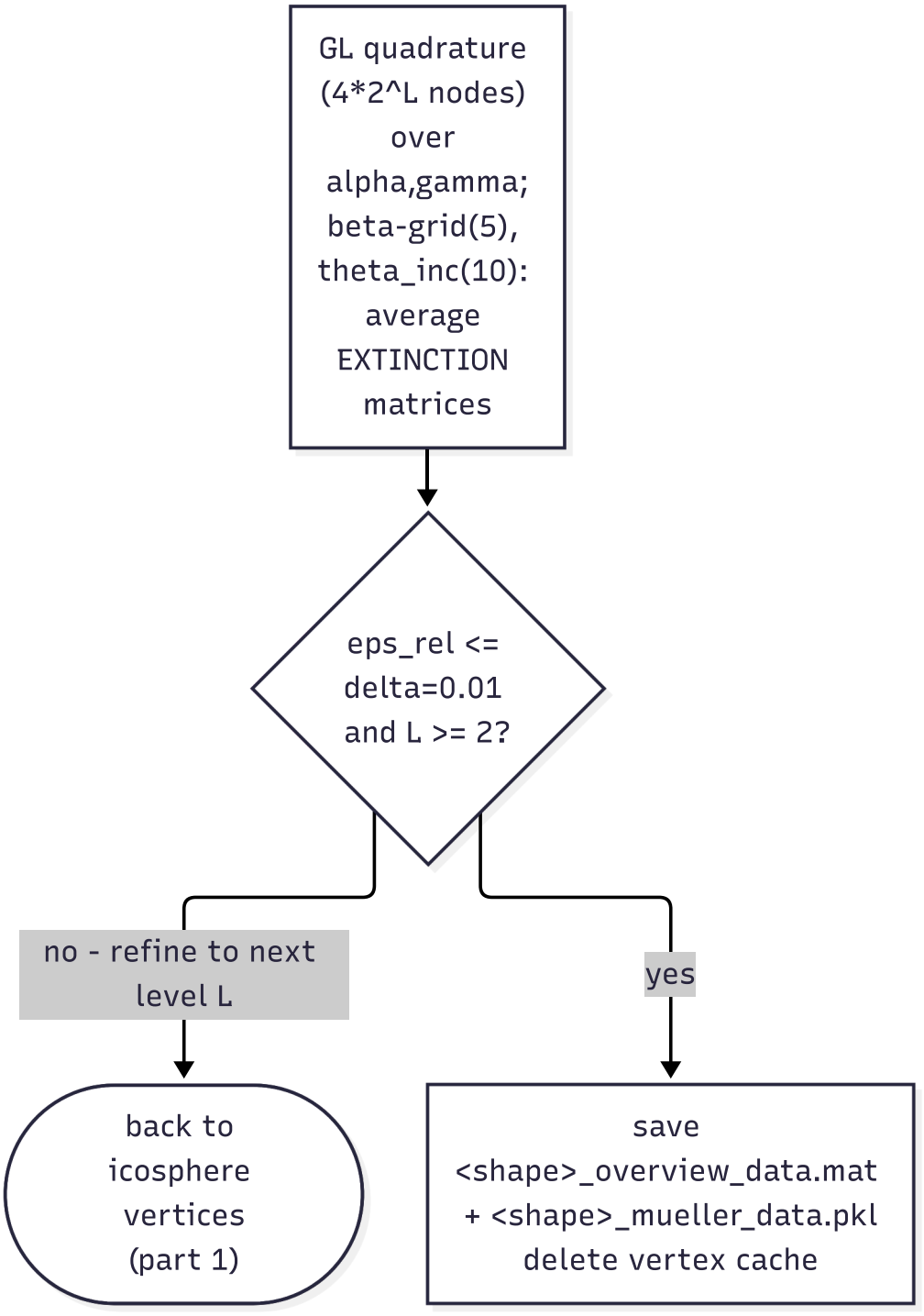
> ⚠ local mode runs thousands of jobs **synchronously in one foreground process**. For anything beyond smoke tests, deselect sizes/frequencies in a probe config or run the vertex loop in parallel (see §3.6).

One job: `adda_sub_control_spharmonics.py` (per T, f, size)

Part 1 — init and the icosphere vertex loop, per refinement level $L = 0\ldots$ (min 3 passes, max `limit_sph` = 4):



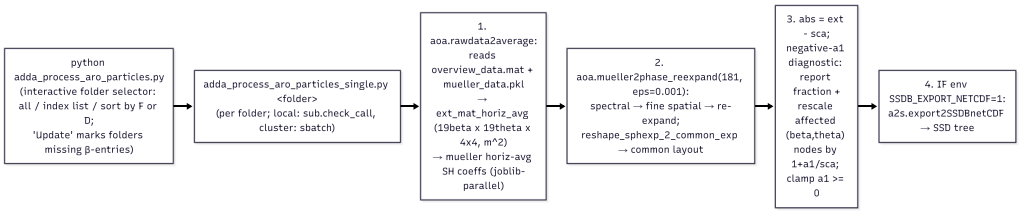
Part 2 — orientation average, refinement loop, outputs:



The mirror-fill transform (φ remap + Stokes signs + SH re-expansion) is sketched in §1.7; the same vertex grid also answers `interpolation_weights_icosahedric_grid` queries during the averaging stage — barycentric interpolation, `aoa.pol_changer` for the Stokes polarization basis.

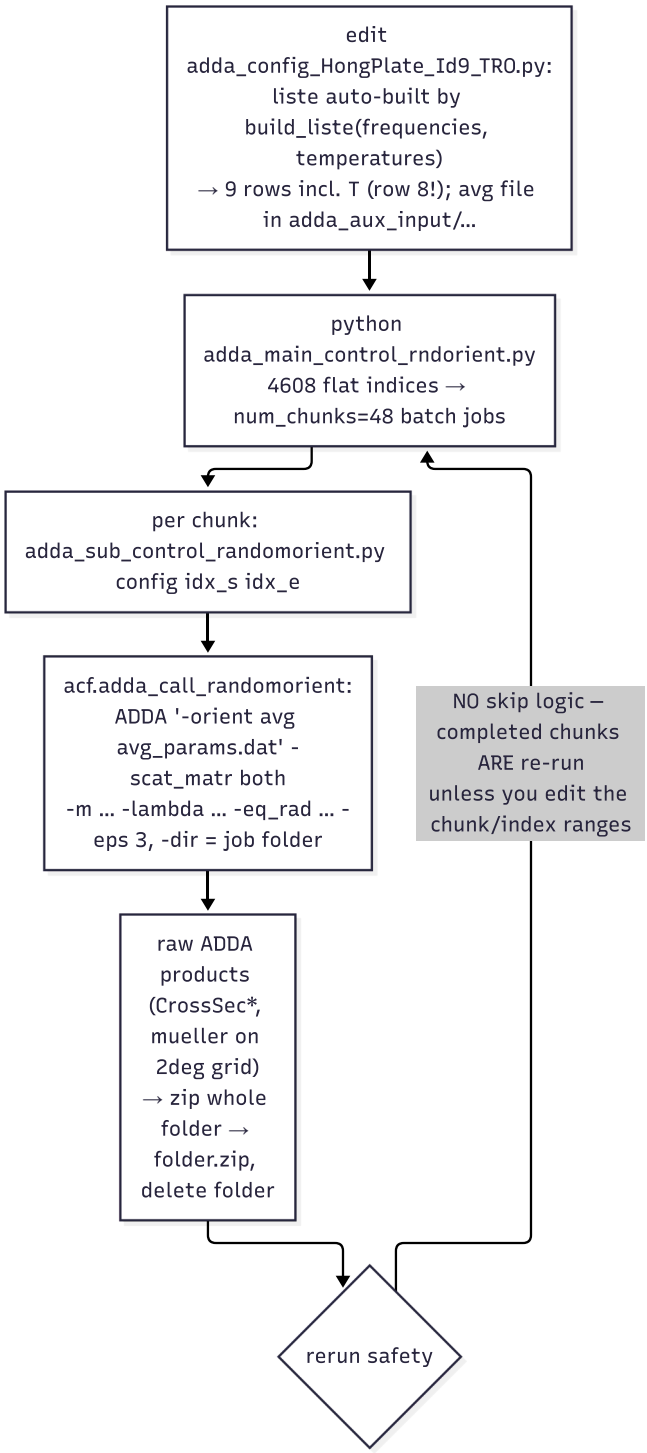
3.3 Post-processing: folder → SSDB NetCDF

Run by hand per habit (or per folder) — this is deliberately decoupled from the solves:



Notes: the 19×19 (β, θ_{inc}) grid and `eps_phase=0.001` are **hard-coded** in `adda_process_aro_particles.py:74–107` (independent of the solve-time 5×10); the SSDB export is **off by default** (`SSDB_EXPORT_NETCDF=1` to enable); the habit processed is env-selected `SSDB_HABIT=<habit>` (default `HexPlaAggCrystal_Id20`). The negative-a1 rescale is a physical-consistency repair: where the spectral C_sca exceeds C_ext the whole phase matrix at that node is scaled down; check the printed diagnostics — a large `frac_of_neg_a1` means the truncation error `eps_sphexp/eps_phase` is fighting you.

3.4 TRO production run



Gate artifacts kept in `analysis/`: `phase5_tro_beta_gate.py/.log` (β-fold equivalence), `phase5_tro_probe.log` (ADDA accepts the avg files in 1.2/1.4/1.5).

3.5 Local (macOS) runbook

Python env — mamba env `DDA` (py3.12): conda-forge `numpy scipy netcdf4 joblib matplotlib fftw pkg-config`; pip `typhon==0.10.0 shtns` (shtns: no osx-arm64 conda build; native wheel — rebuild with `pip install --no-build-isolation --no-deps shtns` while fftw+pkg-config are present). Never downgrade typhon (0.8.0 is broken on 3.12; `ty.arts.xml` is gone — Mueller artifacts are **pickle** files now).

ADDA build (works identically for `adda_1.2`, `adda_1.4`, `adda/`; never run the checked-in Linux ELF binaries under `adda_1.2/src/`):

```
cd adda_1.4/src && make cleanseq                # stale Linux .d files break make otherwise
make seq "OPTIONS=NO_FORTRAN" \
  FFTW3_INC_PATH=$CONDA_PREFIX/../envs/DDA/include \
  FFTW3_LIB_PATH=$CONDA_PREFIX/../envs/DDA/lib
# make links WITHOUT rpath → relink with rpath so dyld finds libfftw3.3.dylib:
cd seq && gcc -o adda *.o -L<env>/lib -Wl,-rpath,<env>/lib -lm -lfftw3
```

(NO_FORTRAN is fine: only IGT/surface modes need Fortran; the pipeline uses `-int defint` default.)

Running a smoke job locally:

```
mamba activate DDA
cd adda_python_scripts
python adda_main_control.py            # pick the config index when prompted
# cross-version experiments without editing configs:
ADDA_BINARY=../adda_1.4/src/seq/adda ADDA_VERSION=1.4.0 python adda_main_control.py
```

The config `local` branch maps `prefix` → `adda_python_scripts/`, `prefix_data` →repo root, so results appear under `./adda_results/…`. Keep probes small: one config with 1 T, 1–2 frequencies, 1–2 sizes (see `adda_config_phase*_…py` for probe patterns). `force_new=True` forces recompute; otherwise completed jobs are skipped.

3.6 HPC runbook (Mistral / Thunder)

- Config hostname branch sets prefixes (`/work/um0878/…`, `/scratch/uni/u237/…`), `ramdrive=/dev/shm…`, Slurm `Partition`, `ncores`; submission goes through the site-specific `sbatch_simple*` wrappers (paths hard-coded in the main controls).
- Cluster configs still default to **ADDA 1.2 MPI** (`adda_1.2/src/mpi/adda_mpi` via `srun -n N`); the production decision is **1.4.0** — build an MPI 1.4 binary on the cluster and flip the triple, or export `ADDA_BINARY` / `ADDA_VERSION`.
- **Parallel job structure (Phase-4b recommendation, report §4b.3):** the per-vertex ADDA solves are process-level parallel and single-core-efficient — pool *N_cores* single-core `adda` invocations inside each (T,f,size) job rather than one MPI-parallel solve; keep the adaptive icosphere refinement sequential (it is a decision loop, not a loop nest).
- Cost anchors for planning: see §4.4 backlog table and `fig07`; the 1.4/1.5 trees also accept the old CLI subset unchanged (report §1.3 compatibility audit) except the `-orient avg` + grid-flag conflict (§1.5.5).

Part 4 — Validation & quality gates

4.1 Golden baselines (`baselines/`)

The frozen regression reference: 6 ADDA cases mirroring the exact CLI pattern of `acf.adda_call` (`-orient 0 0 0`, `-prop 1 0 0`, `-eps 3`, Matzler ice at 230 K): sphere (independent Mie check), oblate ellipsoids x=0.1/1/5, HongPlate_Id9 Dveq10 μm at 192/337 GHz. Current golden: **`baselines/adda_1.4.0/`** (regenerated 2026-11-06); `adda_1.2/` archived for provenance.

```
python baselines/make_baseline.py --adda <adda-seq-binary> --out baselines/<tag>
python baselines/check_baselines.py baselines/<tag>     # analytic sanity (sphere/Mie etc.)
```

The gate rule is noise-based, never bit-level. Measured ADDA re-run scatter of *identical commands* at `-eps 3`: 0.0 for four of six cases, $\leq 10^{-11}$ generally, but $8.3 \cdot 10^{-4}$ **for the** $x = 5$ **case** (12 312 dipoles at residual 10^{-3}). Therefore regression tolerances are: 10^{-10} for small cases, $5 \cdot 10^{-3}$ for $x = 5$. Any stack change (numpy/scipy bump, ADDA switch, symmetry fill-in) must land inside those bands. Do not regenerate goldens casually; a new golden requires re-deriving the noise budget.

4.2 Equivalence suites (what was proven, and how)

gate	command / artifact	result
Phase-2 numpy-2 migration	`analysis/phase2_equiv.py` (pre/post pickles)	pipeline bit-identical on numpy 2 / py3.12 / macOS
ADDA 1.2 vs 1.4.0 vs 1.5.0-alpha3	`analysis/phase3_equiv.py`, `phase3_results.json`	ext rel-diff $3 \cdot 10^{-17}$…$2.5 \cdot 10^{-4}$, mueller $\lesssim 5 \cdot 10^{-4}$ (all below the $5 \cdot 10^{-3}$ noise bound; x5 numbers *below 1.2's own scatter*); auto dipole density identical across versions; TRO `-orient avg` CrossSec **bit-identical → production binary: 1.4.0**
mirror machinery (sym vs asym, same config otherwise)	`analysis/phase4_subcontrol_gate.log`, `phase4_aoa_gate.log`	identical within $4.5 \cdot 10^{-12}$ with *closed* mirrors; a non-closed mirror (coarse YZ, closure 0.986) produces $1.5 \cdot 10^{-4}$ field errors — proof the closure gate is load-bearing
half-range β/θ reuse hypothesis	`phase4b_halfrange_check.py/.log`	**rejected**: $2.3 \cdot 10^{-3}$…$7.2 \cdot 10^{-2}$ discrepancy vs 10^{-11} noise
TRO β-fold (0–90° avg file vs full range)	`phase5_tro_beta_gate.py/.log`	CrossSec $\lesssim 1.35 \cdot 10^{-5}$ (= Romberg tol), Mueller $\lesssim 10^{-13}$ → PASS
ADDA cost/accuracy	`phase4b_adda_benchmark.py`, `…_bench_results.json`	fig. 04; error budget discretization-dominated

Harness lessons that produced *false* failures — remember them when writing new gates: auto dpl is **quantized** (a 10^{-8} λ typo changes N_dipoles and breaks comparisons — compute λ/eq_rad exactly like `make_baseline.py`), and a scalar `theta=90.0` emits `z=6.12e-17` instead of exact `-prop 1 0 0`, rotating the φ reference (up to $2 \cdot 10^{-2}$ Mueller difference at x=5 — use the vector branch).

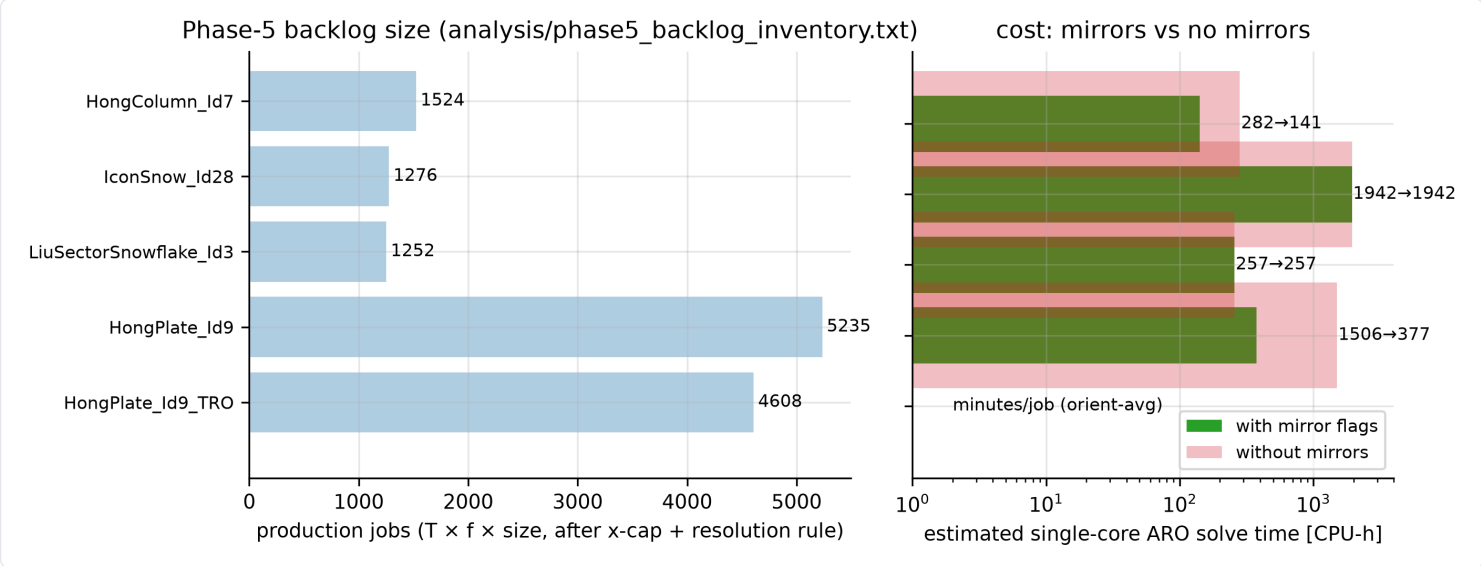
4.3 Where the accuracy lives (error budget recap)

$$\varepsilon_{\text{total}} \approx \underbrace{\varepsilon_{\text{disc}}}_{\text{fmin class}/dpl_{\text{eff}}} + \underbrace{\varepsilon_{\text{orient}}}_{\delta=0.01,\ \geq 3 \text{ levels}} + \underbrace{\varepsilon_{\text{SH}}}_{\text{eps_sphexp}=0.005,\ \text{phase}:10^{-3}} + \underbrace{\varepsilon_{\text{solver}}}_{\text{-eps 3}:\ 10^{-11}\ldots 8\cdot 10^{-4}} + \underbrace{\varepsilon_{\text{fp}}}_{\text{float32/complex64}}$$

with the **discretization term dominant** (design margin $\geq 10^{-3}$).

If a consumer complains about smoothness/positivity (negative absorption vector a1), look at the SH truncation terms and the rescale diagnostics (§3.3), not at ADDA.

4.4 Production backlog (as of Phase 5)



config	jobs	mirrors	est. single-core solve-time
HongPlate_Id9 (ARO, full 35-ch grid)	5235	XY+XZ (25 % solves)	377 CPU-h (unmirrored: 1506)
HongColumn_Id7	1524	YZ (50 %)	141 CPU-h (282)

config	jobs	mirrors	est. single-core solve-time
IconSnow_Id28	1276	none	1942 CPU-h
LiuSectorSnowflake_Id3	1252	none	257 CPU-h
HongPlate_Id9_TRO	4608	(β-folded avg)	minutes per job

≈ **2700 single-core CPU-h** total. Runbook: local mode only for probes; production goes through the sbatch path with 1.4.0-MPI once built on the cluster (§3.6); ARO post-processing/export per §3.3. Inventory reproducibility: `analysis/phase5_backlog_inventory.py` (prints the table above and re-derives the job list with the exact control-selection logic).

Known open items (from report Phase 5, §5.7)

- Cluster MPI build of ADDA 1.4 (until then cluster runs use 1.2 defaults).
- TRO SSDB export path (currently `-orient avg` output is raw ADDA zips; a parser → NetCDF step is still to be specified).
- `sym_Zobj` parked (§1.7); half-range reuse rejected; GL-node β/θ grids rejected (no solves saved) — do not reopen without new evidence.

Appendix A — ADDA command-line tokens used

Full assembled command lines (verified from `acf.adda_call` / the TRO pair):

ARO solve (per icosphere vertex):

```
[launcher] <adda_binary> -shape read <file.adda> \
    -store_scat_grid -scat_grid_inp <scat_params.dat> -scat_matr both \
    -m <Re(m)> <Im(m)>  -lambda <µm>  -eq_rad <µm, equivalent-volume RADIUS> \
    -orient <α0> <β0> <γ0>   -prop <vx vy vz (icosphere vertex unit vector)> \
    -dir <tmpdir>/temp  -eps <3>
```

TRO solve (per (f, size, T) — orientation averaging inside ADDA):

```
[launcher] <adda_binary> -shape read <file.adda> \
    -scat_matr both  -orient avg <avg_params.dat> \
    -m … -lambda … -eq_rad …  -dir <job folder>  -eps <3>
```

token	note
`-eq_rad`	equivalent-volume radius (μm) — `Sizes` holds *diameter*, the sub-control halves it; N_dipoles comes from the shape file, ADDA only rescales
`-orient α β γ`	zyz Euler degrees (production 0 0 0); the *beam* direction comes from `-prop`
`-prop`	incidence unit vector; **always pass the vector branch** (a scalar 90° emits z=$6.12 \cdot 10^{-17}$ and rotates the φ reference)
`-store_scat_grid -scat_grid_inp`	181×361° grid from `scat_params.dat`
`-scat_matr both`	amplitude + Mueller on the grid
`-eps 3`	residual 10^{-3}; noise floor §1.1
removed from configs	`-Csca`, `-alldir_inp` (never parsed; 1.4/1.5 flag-combination traps)
version caveat	`-orient avg` cannot combine with `-scat_grid` / `-Csca` / `-alldir` in 1.4/1.5

Parsing contract: `ampl_scatgrid` 1 header line, forward row first, column order s22, s11, s12, s21 (pairs real/imag); `mueller_scatgrid` 1 header line, cols θ, φ, then 16 elements row-major — **all positional**, re-verify on any ADDA upgrade.

Appendix B — the .dat input files

`scat_params.dat` (`adda_python_scripts/` , handed via `-scat_grid_inp`): `global_type=grid` , θ range N=181 (0–180°), φ range N=361 (0–360°) — 1° resolution; the `phi_integr` section belongs to ADDA's internal integration (unused by us since `-Csca` was removed).

`avg_params.dat` (`adda_aux_input/<habit>/` , TRO): per-Euler-angle Romberg spec `min/max/Jmin/Jmax/eps/equiv/periodic` (zyz). Production file for the TOR grid folds **β to 0–90°** using the XY-mirror closure (equivalence-gated $\leq 1.35 \cdot 10^{-5}$, §1.5.5); the full-range twin `avg_params_fullrange.dat` exists as the gate reference. Rule of thumb from the file comments: axysymmetrical particles → `gamma max=0` ; equivalence planes → fold the range.

Appendix C — SSDB NetCDF layout

Filename: `Freq{f:08.3f}GHz_T{T:03.0f}K.nc` , one per (β, size, f, T), under `SSD/<orientation_type>/<medium>/<crystal_type>/<pristine_type>/<habit>/AzimuthallyRandom_beta{β:03.0f}deg/<Dmax…Dveq…Mass…>/` ; root attrs `date` , `version='beta_0.9'` ; groups `SingleScatteringData` , `ShapeData` , `CalculationData` .

dimension	length	meaning
`sph_coeffs_scat`	common SH layout size	number of shtns coefficients of the phase-matrix re-expansion
`aa_inc`	1	azimuth of incidence (fixed 0°)
`za_inc`	19	zenith incidence-angle axis (°)
`phaMatElem` / `extMatElem` / `absVecElem`	16 / 3 / 2	element selectors (ext: K11, K12, K33; abs: 2 elems)
`scatMat_row` / `scatMat_Col`	4 / 4	Stokes matrix indices

`SingleScatteringData` variables: `frequency` (Hz), `temperature` (K), `za_inc` , `aa_inc` , index maps, **`phaMat_data_real` + `phaMat_data_imag`** (float32, $\mathbf{m^2}$, zlib; dims phaMatElem ×aa_inc ×za_inc ×sph_coeffs_scat), `extMat_data` ($\mathbf{m^2}$), `absVec_data` ($\mathbf{m^2}$).

`ShapeData` (0-d): `diameter_max` , `diameter_vol_eq` , `diameter_area_eq_aerodynamical` , `mass` , `dpl` , `N_dipoles` , `refrIndex_real/imag` , `alpha` , `beta` , `gamma` + attrs (`shape_file` , `refrIndex_model='Maetzler2006'` , `habit_id` , `medium` , …).

`CalculationData` (attrs): `method='DDA'` , `software='ADDA'` , `software_version` , `system` , `n_cores` , `date_completion` , `ADDA_eps` , `ADDA_scatParam_file` , `ADDA_avgParam_file` , `Refinement_delta` , `Maximum allowed refinement level` , `Mueller_sph_eps` , `Phase_sph_eps` , `sym_XYobj/XZobj/YZobj` , shtns version string. Missing config keys degrade to `'n/a'` (`getattr(ac, …, 'n/a')`) — never crash the export.

Appendix D — Gotcha catalog

Numbered for reference; several carry the bug-fix that neutralized them.

1. **Linux ELF binaries in `adda_1.2/src/seq|mpi`** — cluster-built, run nowhere locally. Always rebuild (`make seq` , NO_FORTRAN + FFTW rpath, §3.5); `make cleanseq` first or stale Linux `.d` files abort the build.
2. **Output parsing is positional** (`ampl_scatgrid` col order s22,s11,s12,s21!) — re-verify on ADDA upgrades.
3. **ADDA reruns are not bit-reproducible** at `-eps 3` (up to $8.3 \cdot 10^{-4}$ at x=5) → all gates noise-based.
4. **`-orient avg` + `-scat_grid` / `-Csca` / `-alldir` rejected** by ADDA 1.4/1.5.
5. **typhon must stay 0.10.0**; `ty.arts.xml` is gone — internal Mueller artifacts are **pickle** (`*_mueller_data.pkl`); idempotency check accepts legacy `.xml` names too.
6. **shtns on osx-arm64**: conda-forge has no build; rebuild wheel in-env (`--no-build-isolation --no-deps`) against the env's fftw.
7. **numpy-2 migration done** (Phase 2): `np.nan` , `np.complex128` , `trapezoid` , `np.tile` ; `posix_spawn.py` has a macOS subprocess branch. Never reintroduce old aliases.
8. **Config globals are exec-injected** — renaming a config global breaks the drivers silently; keep the field contract (§3.1).
9. **Deprecated, do not extend**: `adda_sub_control_gridref.py` , `acf.romberg_*` , `adda_orientational_avg_spharmonics_for_testing.py` , `old_configs/` , `SSD_scripts_deprecated/` .
10. **TRO `liste` must have 9 rows** (row 8 = temperature). With 8 rows the folder name gets the flat index as `T####K` — now guarded by a loud warning; rebuild old-style configs instead.
11. **TRO has no idempotency**: chunk ranges are re-run unconditionally; TRO `deq` folder field is the eq **radius** (legacy).

12. **Mirror flags from the coarsest class only**; closure <1.000 fill-in = real shape error (~$1.5 \cdot 10^{-4}$), not noise. Re-run `analysis/phase4_symmetry.py` when adding or regenerating shape files.
13. **f < 192 GHz in the frequency list runs** (coarsest class, grossly over-resolved) — the `=np.array([5,])*1e9` stub configs are pointless work, and `analysis/inventory.py`'s per-file rule counts them as zero (mismatch with the live control's behavior at low f — report §1.7 wording refers to the replica).
14. `-prop` **scalar vs vector branch**: always pass the unit vector (scalar 90° ⇒ z=$6.12 \cdot 10^{-17}$ ⇒ φ-reference rotation ⇒ $2 \cdot 10^{-2}$ Mueller artifacts in comparisons).
15. `export2SSDBnetCDF` **is off by default** (`SSDB_EXPORT_NETCDF=1`) and habit-selected by `SSDB_HABIT` ; the 19×19 grid + `eps_phase=0.001` are hard-coded in `adda_process_aro_particles.py` .
16. **Local mode is synchronous** — `adda_main_control.py` on the Mac runs jobs one-by-one in the foreground; use probe configs, and pool single-core ADDA solves if you parallelize (§3.6).
17. **Per-size class availability is not guaranteed complete** (e.g. HexPlaAgg 197 μm lacks the 192/337/457 classes); and for small particles the five classes are degenerate copies (identical N_dipoles).
18. **Vertex-cache is transient**: `<shape>_temp_data_vertex_*.nc` deleted at job completion; killing a job mid-way leaves it (useful), completing wipes it (startover of a *single vertex* re-solves).
19. **ZIPPING=False** silently deletes the auto-created vertex zip (ADDA raw output is not kept); set `ZIPPING=True` when debugging a solve.
20. **No version control was present historically** — repo is a git checkout now; still: results/intermediates live outside the repo (`adda_results/…`), and old HPC results are unrecoverable.

References

- **Brath, M., Ekelund, R., Eriksson, P., Lemke, O., Buehler, S. A. (2020)**: Microwave and submillimeter wave scattering of oriented ice particles, *Atmos. Meas. Tech.* **13**, 2309–2333, https://doi.org/10.5194/amt-13-2309-2020. — **The main methodology article for everything ARO in this repo**: TRO/ARO definitions, zyz Euler angles, the ARO averaging approach (DDA solves + own orientation averaging over α, γ), the (β, θ_{inc}) output coordinates, and the RT motivation (ICI, GMI 166 GHz polarization). Their ARO database: zenodo 3463003.
- **Eriksson, P., Ekelund, R., Mendrok, J., Brath, M., Lemke, O., Buehler, S. A. (2018)**: A general database of hydrometeor single scattering properties at microwave and sub-millimetre wavelengths, *Earth Syst. Sci. Data* **10**, 1301–1326, https://doi.org/10.5194/essd-10-1301-2018. — The published ARTS SSPDB (TRO) paper: source of the habit-ID numbering and shape-file lineage (`*_Id<n>` folder names), the 35-channel frequency selection, the 190/230/270 K temperature set, the Mätzler-2006 refractive-index choice, and the SSDB NetCDF format the export targets (zenodo 1175572, tooling: `arts-sspdb-interface/`).
- Yurkin & Hoekstra 2011, *JQSRT* 112, 2234–2247 (ADDA); ADDA manual: `adda/doc/manual.pdf` , `adda_1.2/doc/manual.pdf` .
- Mätzler 2006, dielectric function of ice (implemented in `acf.eps_ice_matzler06`).
- Mishchenko 2002 (*Light Scattering by Nonspherical Particles*) — extinction-matrix construction used in `acf.extinction_matrix` .
- ARTS Microwave SSP Database: https://doi.org/10.5281/zenodo.1175572 (browse/import tooling: `arts-sspdb-interface/`).
- Project working documents: `plans/DDA-modernization-plan.md` (approved plan), `plans/DDA-modernization-report.md` (findings per phase — **read before code changes**), `baselines/README.md` .

Generated with the repo state of 2026-11-06. Figures regenerate via `docs/make_figures.py` ; this file plus `docs/DDA-manual.html` are produced from the same source.